OmniCode: Chapter 12: Track 2 - Lexicon: Number Primitive Type

Dual-Rail Polar Planes, 32-Trit Word & Zero-Sum ALU

CHAPTER IDENTIFIER
CH 12

NOTEBOOKLM WORD COUNT
11,893 words

CATALOGED CHAPTER FILES
11 files (2,364 lines)

GENERATION DATE
2026-10-16 20:36:04

EPISTEMIC COVENANT BEDROCK & IMMUTABLE LANDMARKS

"And he gave unto Moses... two tables of testimony, tables of stone, written with the finger of God." — Exodus 31:18

"Prepare thy work without, and make it fit for thyself in the field; and afterwards build thine house." — Proverbs 24:27

"Remove not the ancient landmark, which thy fathers have set." — Proverbs 22:28

GOOGLE NOTEBOOKLM GROUNDING FOCUS & LEARNING PROMPT

- **Chapter Focus:** *"How does dual-rail balance enable O(1) rail-swap negation?"*
- **Architectural Rule:** All definitions conform to 4-block structure & spatial ruler.

CHAPTER MANIFEST & INDEX (11 FILES)

#	FILE PATH	TYPE	LINES	WORDS	SIZE (KB)
1	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/number/README.md**	MD	135	840	5.8
2	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/number/transpiled/interpreted/lua/number.lua**	LUA	347	1,746	15.0
3	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/number/transpiled/interpreted/lua/tests/test_number.lua**	LUA	203	876	8.6
4	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/number/transpiled/compiled/c/Makefile**	TEXT	18	36	0.4
5	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/number/transpiled/compiled/c/compile_flags.txt**	TXT	7	6	0.1
6	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/number/transpiled/compiled/c/test_number_bin**	TEXT	1	5	29.6
7	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/number/transpiled/compiled/c/tests/test_number.c**	C	276	1,205	12.0
8	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/number/transpiled/compiled/c/include/number.h**	H	413	1,949	14.9
9	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/number/transpiled/compiled/c/src/number.c**	C	449	1,737	15.0
10	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/number/interpreted/number.tape**	TAPE	121	899	6.3
11	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/number/compiled/number.omni**	OMNI	394	2,594	21.0

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/number/README.md

MD | 135 LINES | 840 WORDS | 5.8 KB

```
# Invariant Type: < Number >

Welcome to the documentation for **Number**, the foundational scalar measurement and arithmetic invariant type in OmniCode!

---

## 1. What is a Number in OmniCode?

In conventional programming, a number is typically just a fixed binary sequence (like a 32-bit or 64-bit integer) that wraps around or overflows when limits are exceeded.

In **OmniCode**, a Number is an **Invariant Type**:

- It represents the **cosmic order of measure and quantity**.
- **Witness:**
  > *"He telleth the number of the stars; he calleth them all by their names."*
  > — **Psalm 147:4 KJV**
- **Semantics:** `{ ScalarQuantity :: DualRailBitPlane :: ZeroSumEquilibrium }`

### Footprint & Capacity Law

- **Defined Space:** Exactly **1 Machine Word**.
- **Dual Substrate:**
  - Physical Silicon Substrate: **64 Bits** (standard 64-bit microprocessor register, `int64_t` / `uint64_t`).
  - Virtual Balanced Substrate: **32 Balanced Trits** ($3^{32} \approx 1.85 \times 10^{15}$ symmetrical states centered at 0).
- **The Core ALU Zero-Sum Equilibrium Law:**
  $$(+1) + (-1) \iff 0$$
  Balance is not just an arithmetic result; it is the fundamental proof of correct execution.

---

## 2. Dual-Rail Polar Bit-Planes on Silicon

To run balanced ternary numbers at maximum speed on standard binary computers, OmniCode packs 32 trits into a native 64-bit integer using **Dual-Rail Bit-Planes**:

| 2-Bit Code | Rail Activation | Trit Value | Meaning |
| :---: | :--- | :---: | :--- |
| `00` | Neither rail active | `0` | **Equilibrium / True Zero** |
| `01` | Positive rail active (bit 0) | `+1` | **Positive Unity** |
| `10` | Negative rail active (bit 1) | `-1` | **Negative Unity** |
| `11` | Both rails active | `NULL` | **Collision Trap / Hardware Fault** |

### Why Dual-Rail is Revolutionary

1. **Instant SIMD Negation:** Negating all 32 trits in a 64-bit word requires no arithmetic addition or two's complement carry chains. It is performed instantaneously by swapping the positive and negative bit-planes in two CPU cycles!
2. **Built-in Safety Trap:** If memory ever corrupts or attempts an impossible simultaneous positive and negative state (`11`), the tripwire automatically trips and rolls back.

---

## 3. Dynamic Packing Topologies

A byte has 8 bits, which can hold balanced trits in different kinematic arrangements depending on how the number is used:

- **`2-2-2-2` (Pure Dense):** Four 2-bit trits per byte (standard compact storage).
- **`2-1-2-1-2` (Palindromic Balance):** Symmetric resting equilibrium with guard channels.
- **`1-2-1-2-2` (Prefix Carry):** Leaves a leading 1-bit channel for fast forward carry propagation.
- **`2-2-1-2-1` (Postfix Carry):** Leaves a trailing channel for reverse borrow propagation.
- **`3-3-2`, `3-2-3`, `2-3-3` (Directional Vectors):** Partitions bits to represent spatial 3D directions (X, Y, Z axes) directly inside a single byte.

---

## 4. Directory Layout & Three Pillars

Following the OmniCode standard, `number/` is structured into three pillars:

```bash
number/
```
```

```
├── README.md               # This architectural guide
├── compiled/               # Pillar 1: High-Level Source Reel
│   └── number.omni         # Canonical OmniCode source code
├── interpreted/            # Pillar 2: Process Ribbon
│   └── number.tape         # Five-roll interpreted execution tape
└── transpiled/             # Pillar 3: Physical Bridges
    ├── compiled/c/         # Freestanding C99/C11 Hardware Backbone
    │   ├── Makefile        # Pure POSIX build script
    │   ├── compile_flags.txt
    │   ├── include/
    │   │   └── number.h    # omni_number_t, topologies, and dual-rail API
    │   ├── src/
    │   │   └── number.c    # Balanced ternary ALU, addition, negation, packing
    │   └── tests/
    │       └── test_number.c # Freestanding C unit test suite
    └── interpreted/lua/    # Dynamic Coroutine Process Ribbon
        ├── number.lua      # Lua tape implementation driven by TapeEngine
        └── tests/
            └── test_number.lua # Lua unit test suite
```

---

5. The 5-Roll Lifecycle

Both `number.tape` (interpreted) and `number.c` (compiled) execute the Number lifecycle in **5 sequential rolls**:

1. **Inscribe:** Takes a raw quantity (e.g. 4096) and encloses it safely inside the 1-word boundary.
2. **Transduce:** Converts the 64-bit integer into 32 balanced trits and packs them into dual-rail bit-planes.
3. **Compare:** Guards the word boundary and confirms that no `11` collision traps exist.
4. **Equilibrium:** Proves the algebraic Zero-Sum law $(+1) + (-1) = 0$ and confirms that double negation restores the exact original value $(-(-X) == X)$.
5. **Sequence:** Commits the number and prepares it to link smoothly into higher-level vectors and arrays.

---

6. How Number Inherits from Universal (`ladder.baton`)

Notice that `number/` contains **no duplicate universal files**.

It inherits the universal foundation cleanly:

- **In C:** `number.h` includes `"omni_base.h"`. The `Makefile` uses `-I../../../../universal/transpiled/compiled/c/include`.
- **In Lua:** `number.lua` loads `tape_engine.lua` from `universal/transpiled/interpreted/lua/`.

---

7. Running Number Tests

You can test Number across both C and Lua environments:

```bash
# Run the C unit tests
cd transpiled/compiled/c
make test

# Run the Lua unit tests
cd ../../interpreted/lua
lua tests/test_number.lua
```

Both test suites confirm:

- Radix-3 balanced ternary conversion and reconstruction
- Dual-rail bit-plane encoding, decoding, and collision detection
- SIMD rail-swap negation involution: `-(-X) == X`
- Dynamic packing and unpacking across all 7 topologies
- The ALU Zero-Sum Equilibrium Law: `(+1) + (-1) <=> 0`
- Safe execution through all 5 lifecycle rolls

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/number/transpiled/interpreted/lua/number.lua

LUA | 347 LINES | 1746 WORDS | 15.0 KB

```lua
-- ================================================================================
-- BLOCK 1: METADATA & PROCESS TAPE MANIFEST
-- ( Process Tape: < Lexicon { Types of Ideas } > & [ Process { Number } ] )
-- ================================================================================
-- ARCHITECTURAL TOPOLOGY:
--   Interprets and executes the atomic lifecycle for Invariant Type < Number >.
--   Acts as dynamic runtime companion driving C backbone `omni_number_t`.
--   - Dual-Rail Polar Bit-Planes: 00=0 (Equilibrium), 01=+1, 10=-1, 11=Trap
--   - Directional Partitions: 3-3-2, 3-2-3, 2-3-3 (3-space minimum for Z)
--   - Witness: Scripture Psalm 147:4 KJV
--     "He telleth the number of the stars; he calleth them all by their names."
--   - Semantics: { DualRailBitPlanes :: KinematicTopologies :: ZeroSumALU }
-- ================================================================================

local ok_te, TapeEngine = pcall(require, "tape_engine")
if not ok_te then
    package.path = package.path ..
        ";../../../../universal/transpiled/interpreted/lua/?.lua"
    TapeEngine = require("tape_engine")
end

local CONSTANTS = {
    WORD_CAPACITY       = 1,
    TRITS_PER_WORD      = 32,
    BITS_PER_WORD       = 64,
    EXEMPLAR_ZERO       = 0,
    EXEMPLAR_UNITY_POS  = 1,
    EXEMPLAR_UNITY_NEG  = -1,
    EXEMPLAR_PARTITION  = 4096,
    BITPLANE_POS_MASK   = 0x5555555555555555,
    BITPLANE_NEG_MASK   = ~0x5555555555555555,
    TRIT_CODE_ZERO      = 0, -- 0b00: Equilibrium / 0
    TRIT_CODE_POS       = 1, -- 0b01: Positive / +1
    TRIT_CODE_NEG       = 2, -- 0b10: Negative / -1
    TRIT_CODE_NULL      = 3  -- 0b11: Collision / NULL trap
}

local PACK_MODES = {
    DENSE_2222        = 0, -- 2-2-2-2: Standard dense storage (4 trits)
    KINEMATIC_21212   = 1, -- 2-1-2-1-2: Symmetric resting equilibrium
    KINEMATIC_12122   = 2, -- 1-2-1-2-2: Prefix carry/overflow channel
    KINEMATIC_22121   = 3, -- 2-2-1-2-1: Postfix carry/overflow channel
    DIRECTIONAL_332   = 4, -- 3-3-2: Directional vector (X:3, Y:3, Z:2)
    DIRECTIONAL_323   = 5, -- 3-2-3: Directional vector (X:3, Y:2, Z:3)
    DIRECTIONAL_233   = 6  -- 2-3-3: Directional vector (X:2, Y:3, Z:3)
}

local function trit_to_2bit(trit)
    if trit == 0 then return CONSTANTS.TRIT_CODE_ZERO
    elseif trit == 1 then return CONSTANTS.TRIT_CODE_POS
    elseif trit == -1 then return CONSTANTS.TRIT_CODE_NEG
    else return CONSTANTS.TRIT_CODE_NULL end
end

local function two_bit_to_trit(code)
    if code == CONSTANTS.TRIT_CODE_ZERO then return 0, true
    elseif code == CONSTANTS.TRIT_CODE_POS then return 1, true
    elseif code == CONSTANTS.TRIT_CODE_NEG then return -1, true
    else return 0, false end -- NULL collision
end

local function pack_trits_dual_rail(trits)
    local packed = 0
    for i = 1, CONSTANTS.TRITS_PER_WORD do
        local t = trits[i] or 0
        local code = trit_to_2bit(t)
        packed = packed | ((code & 0x03) << ((i - 1) * 2))
    end
    return packed
end

local function unpack_trits_dual_rail(packed, out_trits)
    for i = 1, CONSTANTS.TRITS_PER_WORD do
```

```
        local code = (packed >> ((i - 1) * 2)) & 0x03
        local trit, ok = two_bit_to_trit(code)
        if not ok then return false end
        out_trits[i] = trit
    end
    return true
end

local function negate_dual_rail(packed)
    local pos_rails = packed & CONSTANTS.BITPLANE_POS_MASK
    local neg_rails = packed & CONSTANTS.BITPLANE_NEG_MASK
    return ((pos_rails << 1) & CONSTANTS.BITPLANE_NEG_MASK) |
           ((neg_rails >> 1) & CONSTANTS.BITPLANE_POS_MASK)
end

local function has_null_collision(packed)
    local pos_rails = packed & CONSTANTS.BITPLANE_POS_MASK
    local neg_rails = (packed >> 1) & CONSTANTS.BITPLANE_POS_MASK
    return (pos_rails & neg_rails) ~= 0
end

local function pack_byte(trits, mode)
    mode = mode or PACK_MODES.DENSE_2222
    local b = 0
    if mode == PACK_MODES.DENSE_2222 then
        for i = 1, 4 do
            local code = trit_to_2bit(trits[i] or 0)
            b = b | ((code & 0x03) << ((i - 1) * 2))
        end
    elseif mode == PACK_MODES.KINEMATIC_21212 then
        b = b | (trit_to_2bit(trits[1] or 0) & 0x03)
        b = b | ((trit_to_2bit(trits[2] or 0) & 0x01) << 2)
        b = b | ((trit_to_2bit(trits[2] or 0) & 0x03) << 3)
        b = b | (((trit_to_2bit(trits[2] or 0) >> 1) & 0x01) << 5)
        b = b | ((trit_to_2bit(trits[3] or 0) & 0x03) << 6)
    elseif mode == PACK_MODES.KINEMATIC_12122 then
        b = b | ((trit_to_2bit(trits[1] or 0) & 0x01) << 0)
        b = b | ((trit_to_2bit(trits[1] or 0) & 0x03) << 1)
        b = b | (((trit_to_2bit(trits[2] or 0) >> 1) & 0x01) << 3)
        b = b | ((trit_to_2bit(trits[2] or 0) & 0x03) << 4)
        b = b | ((trit_to_2bit(trits[3] or 0) & 0x03) << 6)
    elseif mode == PACK_MODES.KINEMATIC_22121 then
        b = b | ((trit_to_2bit(trits[1] or 0) & 0x03) << 0)
        b = b | ((trit_to_2bit(trits[2] or 0) & 0x03) << 2)
        b = b | ((trit_to_2bit(trits[3] or 0) & 0x01) << 4)
        b = b | ((trit_to_2bit(trits[3] or 0) & 0x03) << 5)
        b = b | (((trit_to_2bit(trits[3] or 0) >> 1) & 0x01) << 7)
    elseif mode == PACK_MODES.DIRECTIONAL_332 then
        local c1 = trit_to_2bit(trits[1] or 0) & 0x03
        local c2 = trit_to_2bit(trits[2] or 0) & 0x03
        local c3 = trit_to_2bit(trits[3] or 0) & 0x03
        b = b | (c1 << 0) | (c2 << 3) | (c3 << 6)
    elseif mode == PACK_MODES.DIRECTIONAL_323 then
        local c1 = trit_to_2bit(trits[1] or 0) & 0x03
        local c2 = trit_to_2bit(trits[2] or 0) & 0x03
        local c3 = trit_to_2bit(trits[3] or 0) & 0x03
        b = b | (c1 << 0) | (c2 << 3) | (c3 << 5)
    elseif mode == PACK_MODES.DIRECTIONAL_233 then
        local c1 = trit_to_2bit(trits[1] or 0) & 0x03
        local c2 = trit_to_2bit(trits[2] or 0) & 0x03
        local c3 = trit_to_2bit(trits[3] or 0) & 0x03
        b = b | (c1 << 0) | (c2 << 2) | (c3 << 5)
    end
    return b & 0xFF
end

local function unpack_byte(byte_val, mode, out_trits)
    mode = mode or PACK_MODES.DENSE_2222
    if mode == PACK_MODES.DENSE_2222 then
        for i = 1, 4 do
            local code = (byte_val >> ((i - 1) * 2)) & 0x03
            local t, ok = two_bit_to_trit(code)
            if not ok then return false end
            out_trits[i] = t
        end
    elseif mode == PACK_MODES.KINEMATIC_21212 then
        local t1, ok1 = two_bit_to_trit(byte_val & 0x03)
        local t2, ok2 = two_bit_to_trit((byte_val >> 3) & 0x03)
```

```
        local t3, ok3 = two_bit_to_trit((byte_val >> 6) & 0x03)
        if not (ok1 and ok2 and ok3) then return false end
        out_trits[1], out_trits[2], out_trits[3], out_trits[4] = t1, t2, t3, 0
    elseif mode == PACK_MODES.KINEMATIC_12122 then
        local t1, ok1 = two_bit_to_trit((byte_val >> 1) & 0x03)
        local t2, ok2 = two_bit_to_trit((byte_val >> 4) & 0x03)
        local t3, ok3 = two_bit_to_trit((byte_val >> 6) & 0x03)
        if not (ok1 and ok2 and ok3) then return false end
        out_trits[1], out_trits[2], out_trits[3], out_trits[4] = t1, t2, t3, 0
    elseif mode == PACK_MODES.KINEMATIC_22121 then
        local t1, ok1 = two_bit_to_trit((byte_val >> 0) & 0x03)
        local t2, ok2 = two_bit_to_trit((byte_val >> 2) & 0x03)
        local t3, ok3 = two_bit_to_trit((byte_val >> 5) & 0x03)
        if not (ok1 and ok2 and ok3) then return false end
        out_trits[1], out_trits[2], out_trits[3], out_trits[4] = t1, t2, t3, 0
    elseif mode == PACK_MODES.DIRECTIONAL_332 then
        local t1, ok1 = two_bit_to_trit((byte_val >> 0) & 0x03)
        local t2, ok2 = two_bit_to_trit((byte_val >> 3) & 0x03)
        local t3, ok3 = two_bit_to_trit((byte_val >> 6) & 0x03)
        if not (ok1 and ok2 and ok3) then return false end
        out_trits[1], out_trits[2], out_trits[3], out_trits[4] = t1, t2, t3, 0
    elseif mode == PACK_MODES.DIRECTIONAL_323 then
        local t1, ok1 = two_bit_to_trit((byte_val >> 0) & 0x03)
        local t2, ok2 = two_bit_to_trit((byte_val >> 3) & 0x03)
        local t3, ok3 = two_bit_to_trit((byte_val >> 5) & 0x03)
        if not (ok1 and ok2 and ok3) then return false end
        out_trits[1], out_trits[2], out_trits[3], out_trits[4] = t1, t2, t3, 0
    elseif mode == PACK_MODES.DIRECTIONAL_233 then
        local t1, ok1 = two_bit_to_trit((byte_val >> 0) & 0x03)
        local t2, ok2 = two_bit_to_trit((byte_val >> 2) & 0x03)
        local t3, ok3 = two_bit_to_trit((byte_val >> 5) & 0x03)
        if not (ok1 and ok2 and ok3) then return false end
        out_trits[1], out_trits[2], out_trits[3], out_trits[4] = t1, t2, t3, 0
    end
    return true
end

local NumberTape = TapeEngine.create_tape({
    metadata = {
        who     = "OmniCode Lexicon",
        what    = "Number --> MEASURE { Quantity }",
        when    = "Arithmetic Event",
        where   = { number_word = 1 },
        why     = "Inscribe scalar number invariant and arithmetic equilibrium",
        how     = "+1 : PROCEED",
        witness = "Psalm 147:4 KJV"
    },
    constants = CONSTANTS,
    pack_modes = PACK_MODES,
    rolls = {
        -- ------------------------------------------------------------------
        -- ROLL 1: QUANTITY INSCRIPTION (Outside-in imposition onto C word)
        -- ------------------------------------------------------------------
        {
            name = "Inscribe",
            action = function(ctx)
                ctx.c_word.scalar = ctx.quantity
                ctx.c_word.is_enclosed = true
                ctx.c_word.pack_mode = ctx.pack_mode or PACK_MODES.DENSE_2222
                return TapeEngine.STATE_PROCEED
            end
        },

        -- ------------------------------------------------------------------
        -- ROLL 2: DUAL-SUBSTRATE TRANSDUCTION (64 Bits <=> 32 Balanced Trits)
        -- ------------------------------------------------------------------
        {
            name = "Transduce",
            action = function(ctx)
                local current = ctx.c_word.scalar
                for i = 1, CONSTANTS.TRITS_PER_WORD do
                    if current == 0 then
                        ctx.c_word.trits[i] = 0
                    else
                        local rem = current % 3
                        current = math.floor(current / 3)
                        if rem == 2 then
                            rem = -1
```

```
                            current = current + 1
                        elseif rem == -2 then
                            rem = 1
                            current = current - 1
                        end
                        ctx.c_word.trits[i] = rem
                    end
                end
                -- Inscribe dual-rail bit-planes (64 bits native)
                local p = pack_trits_dual_rail(ctx.c_word.trits)
                ctx.c_word.packed_dual_rail = p
                return TapeEngine.STATE_PROCEED
            end
        },

        -- ------------------------------------------------------------------
        -- ROLL 3: IDENTITY & BOUND COMPARISON (Guard Single-Word Boundary)
        -- ------------------------------------------------------------------
        {
            name = "Compare",
            action = function(ctx)
                local identity_ok = (ctx.c_word.scalar == ctx.quantity)
                local bounds_ok   = (ctx.c_word.is_enclosed == true)
                local p = ctx.c_word.packed_dual_rail
                local no_null     = not has_null_collision(p)
                local roundtrip   = {}
                local unpack_ok   = unpack_trits_dual_rail(p, roundtrip)
                return TapeEngine.assert_guard(identity_ok and bounds_ok and
                                               no_null and unpack_ok, ctx,
                                               "Number bound or collision")
            end
        },

        -- ------------------------------------------------------------------
        -- ROLL 4: ARITHMETIC EQUILIBRIUM CALCULATION ((+1) plus (-1) <=> 0)
        -- ------------------------------------------------------------------
        {
            name = "Equilibrium",
            action = function(ctx)
                local eq = (1) + (-1)
                local eq_holds = (eq == 0)
                -- Prove SIMD rail-swap involution: -(-x) == x
                local neg = negate_dual_rail(ctx.c_word.packed_dual_rail)
                local neg_neg = negate_dual_rail(neg)
                local rail_involution = (neg_neg == ctx.c_word.packed_dual_rail)
                ctx.equilibrium_proved = eq_holds and rail_involution
                return TapeEngine.assert_guard(ctx.equilibrium_proved, ctx,
                                               "ALU Equilibrium Law violated")
            end
        },

        -- ------------------------------------------------------------------
        -- ROLL 5: SEQUENCE TRANSITION (Number chains toward Vector sequence)
        -- ------------------------------------------------------------------
        {
            name = "Sequence",
            action = function(ctx)
                ctx.vector_ready = true
                return TapeEngine.STATE_PROCEED
            end
        }
    }
})

-- Expose bit-plane and kinematic packing primitives
NumberTape.PACK_MODES             = PACK_MODES
NumberTape.trit_to_2bit           = trit_to_2bit
NumberTape.two_bit_to_trit        = two_bit_to_trit
NumberTape.pack_trits_dual_rail   = pack_trits_dual_rail
NumberTape.unpack_trits_dual_rail = unpack_trits_dual_rail
NumberTape.negate_dual_rail       = negate_dual_rail
NumberTape.has_null_collision     = has_null_collision
NumberTape.pack_byte              = pack_byte
NumberTape.unpack_byte            = unpack_byte

-- ============================================================================
-- BLOCK 2: OPENING BLOCK
-- ( Context Factory & C Word Substrate Mount )
```

```lua
-- ============================================================================

---
-- Mounts the tape head, initialises the C word handle, and arms the tripwire.
-- @param quantity Outside-in scalar integer (defaults to 4096).
-- @param pack_mode Kinematic packing mode (defaults to DENSE_2222).
-- @return Initialized context table.
function NumberTape.open(quantity, pack_mode)
    local p_mode = pack_mode or PACK_MODES.DENSE_2222
    return {
        quantity           = quantity or CONSTANTS.EXEMPLAR_PARTITION,
        pack_mode          = p_mode,
        c_word             = {
            scalar           = 0,
            trits            = {},
            packed_dual_rail = 0,
            pack_mode        = p_mode,
            is_enclosed      = false
        },
        equilibrium_proved = false,
        status             = TapeEngine.STATE_PROCEED,
        tripwire           = TapeEngine.TRIPWIRE_ARMED
    }
end

-- ============================================================================
-- BLOCK 4: CLOSING BLOCK
-- ( Fourfold Detent Commit & Interlock Verification )
-- ============================================================================

---
-- Closes and seals the Number tape execution context.
-- @param ctx Active context handle.
-- @return Quad-State commit status and diagnostic seal message.
function NumberTape.close(ctx)
    return TapeEngine.verify_and_commit(ctx)
end

return NumberTape
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/number/transpiled/interpreted/lua/tests/test_number.lua

LUA | 203 LINES | 876 WORDS | 8.6 KB

```lua
-- ================================================================================
-- BLOCK 1: METADATA & TEST TOPOLOGY
-- ( Unit Test: < Lexicon { Types of Ideas } > & [ Test { Number Tape } ] )
-- ================================================================================
-- ARCHITECTURAL TOPOLOGY:
--   Validates the interpreted process ribbon < Number Tape > in isolation:
--   - Radix-3 balanced ternary transduction (64 bits <=> 32 balanced trits)
--   - Dual-rail polar bit-planes: 00=0 (Equilibrium), 01=+1, 10=-1, 11=Trap
--   - SIMD rail-swap negation: -(-X) == X via bit-plane swap
--   - Collision detection: NULL code (11) trips safety interlock
--   - Dynamic packing topologies: 2-2-2-2, 2-1-2-1-2, 1-2-1-2-2, 2-2-1-2-1,
--     3-3-2, 3-2-3, 2-3-3
--   - Core ALU Zero-Sum Equilibrium Law: (+1) plus (-1) <=> 0
--   - Single-word boundary laws & 4-block lifecycle execution
--   - Witness: Scripture Psalm 147:4 KJV
--     "He telleth the number of the stars; he calleth them all by their names."
--   - Semantics: { DualRailPlanes :: KinematicTopologies :: ZeroSumBalance }
-- ================================================================================

local script_dir = debug.getinfo(1, "S").source:match("@?(.*[/\\])") or "./"
local u_dir = script_dir ..
    "../../../../../universal/transpiled/interpreted/lua/"
package.path = script_dir .. "?.lua;" ..
               script_dir .. "../?.lua;" ..
               u_dir .. "?.lua;" ..
               u_dir .. "tests/?.lua;" ..
               package.path

local TestCommon = require("test_common")
local TapeEngine = require("tape_engine")
local NumberTape = require("number")

local Suite = {}

function Suite.test_number_dual_rail_bitplanes()
    -- 1. Test 2-bit codes: 00=0, 01=+1, 10=-1, 11=trap
    TestCommon.assert_eq(NumberTape.trit_to_2bit(0), 0, "0 must map to 0b00")
    TestCommon.assert_eq(NumberTape.trit_to_2bit(1), 1, "1 must map to 0b01")
    TestCommon.assert_eq(NumberTape.trit_to_2bit(-1), 2, "-1 must map to 0b10")

    local t0, ok0 = NumberTape.two_bit_to_trit(0)
    TestCommon.assert(ok0 and t0 == 0, "0b00 must decode to 0")
    local t1, ok1 = NumberTape.two_bit_to_trit(1)
    TestCommon.assert(ok1 and t1 == 1, "0b01 must decode to +1")
    local t2, ok2 = NumberTape.two_bit_to_trit(2)
    TestCommon.assert(ok2 and t2 == -1, "0b10 must decode to -1")
    local _, ok3 = NumberTape.two_bit_to_trit(3)
    TestCommon.assert(not ok3, "0b11 must report collision / NULL trap")

    -- 2. Test dual-rail packing and unpacking
    local trits = {}
    for i = 1, 32 do
        if i % 3 == 1 then trits[i] = 1
        elseif i % 3 == 2 then trits[i] = -1
        else trits[i] = 0 end
    end
    local packed = NumberTape.pack_trits_dual_rail(trits)
    local unpacked = {}
    local ok = NumberTape.unpack_trits_dual_rail(packed, unpacked)
    TestCommon.assert(ok, "Unpack must succeed on valid trits")
    for i = 1, 32 do
        TestCommon.assert_eq(unpacked[i], trits[i],
                             "Unpacked trit must match original at " .. i)
    end
end

function Suite.test_number_simd_rail_swap_negation()
    local trits = {}
    for i = 1, 32 do
        if i % 3 == 1 then trits[i] = 1
        elseif i % 3 == 2 then trits[i] = -1
        else trits[i] = 0 end
    end
```

```
    local packed = NumberTape.pack_trits_dual_rail(trits)
    local negated = NumberTape.negate_dual_rail(packed)

    local unpacked_neg = {}
    local ok = NumberTape.unpack_trits_dual_rail(negated, unpacked_neg)
    TestCommon.assert(ok, "Unpack negated trits must succeed")
    for i = 1, 32 do
        TestCommon.assert_eq(unpacked_neg[i], -trits[i],
                             "Negated trit must invert sign at " .. i)
    end

    -- Involution: -(-X) == X
    local double_neg = NumberTape.negate_dual_rail(negated)
    TestCommon.assert_eq(double_neg, packed,
                         "Double rail-swap negation must restore original")
end

function Suite.test_number_null_collision_trap()
    -- Valid packed trits have no null collision
    local trits = { 1, -1, 0, 1 }
    local packed = NumberTape.pack_trits_dual_rail(trits)
    TestCommon.assert(not NumberTape.has_null_collision(packed),
                      "Valid trits must not trigger null collision")

    -- Manually inject 11 (code 3) into trit slot 0
    local corrupted = packed | 3
    TestCommon.assert(NumberTape.has_null_collision(corrupted),
                      "Corrupted dual-rail must detect null collision")

    local unpacked = {}
    local ok = NumberTape.unpack_trits_dual_rail(corrupted, unpacked)
    TestCommon.assert(not ok, "Unpacking corrupted dual-rail must fail")
end

function Suite.test_number_dynamic_packing_topologies()
    local modes = {
        NumberTape.PACK_MODES.DENSE_2222,
        NumberTape.PACK_MODES.KINEMATIC_21212,
        NumberTape.PACK_MODES.KINEMATIC_12122,
        NumberTape.PACK_MODES.KINEMATIC_22121,
        NumberTape.PACK_MODES.DIRECTIONAL_332,
        NumberTape.PACK_MODES.DIRECTIONAL_323,
        NumberTape.PACK_MODES.DIRECTIONAL_233
    }
    local test_trits = { 1, -1, 1, -1 }

    for _, mode in ipairs(modes) do
        local byte_val = NumberTape.pack_byte(test_trits, mode)
        local out_trits = {}
        local ok = NumberTape.unpack_byte(byte_val, mode, out_trits)
        TestCommon.assert(ok, "Unpack byte must succeed for mode " .. mode)
        -- Verify first 3 trits match in kinematic / directional modes
        TestCommon.assert_eq(out_trits[1], test_trits[1], "Trit 1 must match")
        TestCommon.assert_eq(out_trits[2], test_trits[2], "Trit 2 must match")
        TestCommon.assert_eq(out_trits[3], test_trits[3], "Trit 3 must match")
        if mode == NumberTape.PACK_MODES.DENSE_2222 then
            TestCommon.assert_eq(out_trits[4], test_trits[4],
                                 "Trit 4 must match in dense mode")
        end
    end
end

function Suite.test_number_transduction()
    local values = { 0, 1, -1, 4096, -4096, 65536 }
    for _, val in ipairs(values) do
        local ctx = NumberTape.open(val)
        local status = NumberTape:execute_all(ctx)
        TestCommon.assert_eq(status, TapeEngine.STATE_PROCEED,
                             "Execution must PROCEED (+1)")
        TestCommon.assert_eq(ctx.c_word.scalar, val,
                             "Context word scalar must match input")
        TestCommon.assert(ctx.c_word.is_enclosed == true,
                          "Context word must be enclosed")
        TestCommon.assert_eq(ctx.equilibrium_proved, true,
                             "ALU Zero-Sum Equilibrium must be proved")
    end
end

```

```
function Suite.test_number_equilibrium_law()
    -- Direct law verification: (+1) + (-1) == 0
    local sum = (1) + (-1)
    TestCommon.assert_eq(sum, 0,
                         "ALU Zero-Sum Equilibrium Law must evaluate to zero")
end

function Suite.test_number_lifecycle_rolls()
    local ctx = NumberTape.open(4096)
    TestCommon.assert_eq(ctx.tripwire, TapeEngine.TRIPWIRE_ARMED,
                         "Tripwire must be ARMED on open")

    local status = NumberTape:execute_all(ctx)
    TestCommon.assert_eq(status, TapeEngine.STATE_PROCEED,
                         "Execution must PROCEED")

    local close_st = NumberTape.close(ctx)
    TestCommon.assert_eq(close_st, TapeEngine.STATE_PROCEED,
                         "Close must seal in PROCEED (+1)")
    TestCommon.assert_eq(ctx.tripwire, TapeEngine.TRIPWIRE_DISARMED,
                         "Tripwire must be DISARMED on close")
end

function Suite.run()
    print("[SUITE: Number Process Ribbon (Lua)]")
    local ok1 = TestCommon.run_test("Dual-Rail Polar Bit-Planes",
                                    Suite.test_number_dual_rail_bitplanes)
    local ok2 = TestCommon.run_test("SIMD Rail-Swap Negation",
                                    Suite.test_number_simd_rail_swap_negation)
    local ok3 = TestCommon.run_test("NULL Collision Trap",
                                    Suite.test_number_null_collision_trap)
    local ok4 = TestCommon.run_test(
        "Dynamic Packing Topologies",
        Suite.test_number_dynamic_packing_topologies
    )
    local ok5 = TestCommon.run_test("Radix-3 Balanced Transduction",
                                    Suite.test_number_transduction)
    local ok6 = TestCommon.run_test("ALU Zero-Sum Equilibrium Law",
                                    Suite.test_number_equilibrium_law)
    local ok7 = TestCommon.run_test("4-Block Lifecycle Rolls",
                                    Suite.test_number_lifecycle_rolls)
    return ok1 and ok2 and ok3 and ok4 and ok5 and ok6 and ok7
end

-- Allow standalone execution
if pcall(debug.getlocal, 4, 1) == false then
    local ok = Suite.run()
    os.exit(ok and 0 or 1)
end

return Suite
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/number/transpiled/compiled/c/Makefile

TEXT | 18 LINES | 36 WORDS | 0.4 KB

```
CC ?= gcc
CFLAGS ?= -Wall -Wextra -std=c11 -Iinclude \
          -I../../../../universal/transpiled/compiled/c/include \
          -I../../../../universal/transpiled/compiled/c/tests

all: test

test: test_number_bin
    ./test_number_bin

test_number_bin: src/number.c tests/test_number.c include/number.h
    $(CC) $(CFLAGS) src/number.c tests/test_number.c -o $@

clean:
    rm -f test_number_bin

.PHONY: all test clean
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/number/transpiled/compiled/c/compile_flags.txt

TXT | 7 LINES | 6 WORDS | 0.1 KB

```
-Iinclude
-I../../../../universal/transpiled/compiled/c/include
-I../../../../universal/transpiled/compiled/c/tests
-Wall
-Wextra
-std=c11
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/number/transpiled/compiled/c/test_number_bin DOC/BINARY 1 LINES 5 WORDS 29.6 KB

[Binary Artifact: test_number_bin (29.6 KB)]

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/number/transpiled/compiled/c/tests/test_number.c

C | 276 LINES | 1205 WORDS | 12.0 KB

```c
/* ============================================================================
 * BLOCK 1: METADATA & TEST TOPOLOGY
 * ( Unit Test: < Lexicon { Types of Ideas } > & [ Test { Number } ] )
 * ============================================================================
 * ARCHITECTURAL TOPOLOGY:
 *   Validates the scalar invariant type < Number > in isolation:
 *   - Bijective round-trip radix-3 transduction (64 bits <=> 32 balanced trits)
 *   - Dual-rail bit-plane packed representation and O(1) rail-swap negation
 *   - Instant O(1) NULL collision trap detection
 *   - Dynamic packing topologies: 2-2-2-2, 2-1-2-1-2, 1-2-1-2-2, 2-2-1-2-1,
 *     3-3-2, 3-2-3, 2-3-3
 *   - ALU Zero-Sum Equilibrium Law: (+1) plus (-1) <=> 0
 *   - Balanced arithmetic addition, subtraction, and inverse cancellation
 *   - Strict single-word boundary laws
 *   - Full 4-block lifecycle execution (open, execute, close)
 *   - Witness: Scripture Psalm 147:4 KJV
 *     "He telleth the number of the stars; he calleth them all by their names."
 *   - Semantics: { DualRailBitPlanes :: KinematicTopologies :: ALUZeroSum }
 * ============================================================================ */

#include "number.h"
#include "test_common.h"

/* ============================================================================
 * BLOCK 2: OPENING CONTEXT & FIXTURE SETUP
 * ============================================================================ */

static int test_number_bijective_roundtrip(void) {
    const int64_t test_values[] = {
        0, 1, -1, 2, -2, 4096, -4096, 65536, -65536, 1000000, -1000000
    };
    const size_t num_values = sizeof(test_values) / sizeof(test_values[0]);

    for (size_t i = 0; i < num_values; ++i) {
        int64_t original = test_values[i];
        omni_number_t n = omni_number_from_scalar(original);

        OMNI_TEST_ASSERT(n.is_enclosed == true, "Number must be enclosed");
        OMNI_TEST_ASSERT(n.scalar == original,
                         "Number scalar must match original");

        /* Verify all 32 trits are in {-1, 0, +1} */
        for (size_t t = 0; t < OMNI_NUM_TRITS_PER_WORD; ++t) {
            OMNI_TEST_ASSERT(n.trits[t] >= -1 && n.trits[t] <= 1,
                             "Trit must be in {-1, 0, +1}");
        }

        /* Verify reconstruction from trits */
        int64_t reconstructed = omni_number_to_scalar(n.trits);
        OMNI_TEST_ASSERT(reconstructed == original,
                         "Reconstructed scalar must equal original value");
    }
    return 0;
}

static int test_number_dual_rail_bitplanes(void) {
    /* Test Zero: All trits 00 -> packed is 0 */
    omni_number_t zero = omni_number_from_scalar(0);
    OMNI_TEST_ASSERT(
        omni_number_trits_is_equilibrium(zero.packed_trits) == true,
                     "Packed zero must be in equilibrium (0ULL)");
    OMNI_TEST_ASSERT(omni_number_trits_has_null(zero.packed_trits) == false,
                     "Packed zero must not have NULL traps");

    /* Test Positive Unity (+1): First trit is 01, others 00 */
    omni_number_t pos_one = omni_number_from_scalar(1);
    OMNI_TEST_ASSERT((pos_one.packed_trits & 0x03) == OMNI_TRIT_PAIR_POS,
                     "First trit of +1 must be 01 (positive rail active)");

    /* Test Negative Unity (-1): First trit is 10, others 00 */
    omni_number_t neg_one = omni_number_from_scalar(-1);
    OMNI_TEST_ASSERT((neg_one.packed_trits & 0x03) == OMNI_TRIT_PAIR_NEG,
                     "First trit of -1 must be 10 (negative rail active)");
```

```
    /* Test Instant Bit-Plane Negation (+1 <-> -1) */
    uint64_t negated_pos = omni_number_trits_negate(pos_one.packed_trits);
    OMNI_TEST_ASSERT((negated_pos & 0x03) == OMNI_TRIT_PAIR_NEG,
                     "Negating +1 packed trits must produce -1 (10)");
    OMNI_TEST_ASSERT(negated_pos == neg_one.packed_trits,
                     "Negated +1 packed word must equal -1 packed word");

    /* Test NULL Trap Detection: artificially inject 11 into trit 0 */
    uint64_t corrupted = pos_one.packed_trits | 0x03;
    OMNI_TEST_ASSERT(omni_number_trits_has_null(corrupted) == true,
                     "Packed word with 11 must be flagged as NULL trap");

    return 0;
}

static int test_number_dynamic_packing_topologies(void) {
    uint8_t out[5];

    /* 1. Test 2-2-2-2 (Dense: 4 fields of 2 bits) */
    uint8_t p_2222 = omni_number_pack_byte(1, 2, 0, 3, 0, OMNI_PACK_2_2_2_2);
    omni_number_unpack_byte(p_2222, out, OMNI_PACK_2_2_2_2);
    OMNI_TEST_ASSERT(out[0] == 1 && out[1] == 2 && out[2] == 0 && out[3] == 3,
                     "2-2-2-2 round-trip unpack must match input");

    /* 2. Test 2-1-2-1-2 (Palindromic Equilibrium: 00/0/00/0/00) */
    uint8_t p_pal = omni_number_pack_byte(3, 1, 2, 0, 1, OMNI_PACK_2_1_2_1_2);
    omni_number_unpack_byte(p_pal, out, OMNI_PACK_2_1_2_1_2);
    OMNI_TEST_ASSERT(out[0] == 3 && out[1] == 1 && out[2] == 2 &&
                     out[3] == 0 && out[4] == 1,
                     "2-1-2-1-2 round-trip unpack must match input");

    /* 3. Test 1-2-1-2-2 (Prefix Carry) */
    uint8_t p_pre = omni_number_pack_byte(1, 3, 0, 2, 1, OMNI_PACK_1_2_1_2_2);
    omni_number_unpack_byte(p_pre, out, OMNI_PACK_1_2_1_2_2);
    OMNI_TEST_ASSERT(out[0] == 1 && out[1] == 3 && out[2] == 0 &&
                     out[3] == 2 && out[4] == 1,
                     "1-2-1-2-2 round-trip unpack must match input");

    /* 4. Test 2-2-1-2-1 (Postfix Carry) */
    uint8_t p_post = omni_number_pack_byte(2, 1, 1, 3, 0, OMNI_PACK_2_2_1_2_1);
    omni_number_unpack_byte(p_post, out, OMNI_PACK_2_2_1_2_1);
    OMNI_TEST_ASSERT(out[0] == 2 && out[1] == 1 && out[2] == 1 &&
                     out[3] == 3 && out[4] == 0,
                     "2-2-1-2-1 round-trip unpack must match input");

    /* 5. Test 3-3-2 (3-Space Directional Partition) */
    uint8_t p_332 = omni_number_pack_byte(7, 5, 2, 0, 0, OMNI_PACK_3_3_2);
    omni_number_unpack_byte(p_332, out, OMNI_PACK_3_3_2);
    OMNI_TEST_ASSERT(out[0] == 7 && out[1] == 5 && out[2] == 2,
                     "3-3-2 round-trip unpack must match input");

    /* 6. Test 3-2-3 (3-Space Directional Partition) */
    uint8_t p_323 = omni_number_pack_byte(6, 3, 5, 0, 0, OMNI_PACK_3_2_3);
    omni_number_unpack_byte(p_323, out, OMNI_PACK_3_2_3);
    OMNI_TEST_ASSERT(out[0] == 6 && out[1] == 3 && out[2] == 5,
                     "3-2-3 round-trip unpack must match input");

    /* 7. Test 2-3-3 (3-Space Directional Partition) */
    uint8_t p_233 = omni_number_pack_byte(3, 7, 4, 0, 0, OMNI_PACK_2_3_3);
    omni_number_unpack_byte(p_233, out, OMNI_PACK_2_3_3);
    OMNI_TEST_ASSERT(out[0] == 3 && out[1] == 7 && out[2] == 4,
                     "2-3-3 round-trip unpack must match input");

    return 0;
}

static int test_number_equilibrium_law(void) {
    /* ALU Equilibrium Law: (+1) plus (-1) <=> 0 */
    bool eq_holds = omni_number_verify_equilibrium();
    OMNI_TEST_ASSERT(eq_holds == true,
                     "Core ALU Zero-Sum Equilibrium Law must hold");
    return 0;
}

static int test_number_addition_and_subtraction(void) {
    omni_number_t a = omni_number_from_scalar(4000);
    omni_number_t b = omni_number_from_scalar(96);
```

```
    omni_number_t sum = omni_number_add(a, b);

    OMNI_TEST_ASSERT(sum.scalar == 4096, "4000 + 96 must equal 4096");
    OMNI_TEST_ASSERT(sum.is_enclosed == true, "Sum must be enclosed");

    /* Test inverse cancellation via subtraction: a - a == 0 */
    omni_number_t zero = omni_number_sub(sum, sum);
    OMNI_TEST_ASSERT(zero.scalar == 0, "4096 - 4096 must equal 0");
    OMNI_TEST_ASSERT(
        omni_number_trits_is_equilibrium(zero.packed_trits) == true,
                    "Zero difference must be in packed equilibrium");

    /* Test negation: -4096 */
    omni_number_t neg_sum = omni_number_neg(sum);
    OMNI_TEST_ASSERT(neg_sum.scalar == -4096, "Negated 4096 must equal -4096");
    OMNI_TEST_ASSERT(omni_number_add(sum, neg_sum).scalar == 0,
                    "Summing with negated counterpart must yield 0");

    return 0;
}

static int test_number_bounds_and_enclosure(void) {
    omni_number_t valid_num = omni_number_from_scalar(100);
    OMNI_TEST_ASSERT(omni_number_verify_bounds(&valid_num) == true,
                    "Valid number must pass bounds check");

    omni_number_t corrupted = valid_num;
    corrupted.is_enclosed = false;
    OMNI_TEST_ASSERT(omni_number_verify_bounds(&corrupted) == false,
                    "Unenclosed number must fail bounds check");

    /* Test null trap tripping */
    omni_number_t trap_num = valid_num;
    trap_num.packed_trits |= 0x03; /* Inject 11 */
    OMNI_TEST_ASSERT(omni_number_verify_bounds(&trap_num) == false,
                    "Number with NULL collision trap must fail bounds check");

    return 0;
}

static int test_number_lifecycle_rolls(void) {
    omni_number_context_t ctx;
    omni_number_open(&ctx, 4096, OMNI_TARGET_ACCUMULATOR);

    OMNI_TEST_ASSERT(ctx.tripwire == OMNI_TRIPWIRE_ARMED,
                    "Tripwire must be ARMED upon open");
    OMNI_TEST_ASSERT(ctx.input_quantity == 4096,
                    "Input quantity must match 4096");

    /* Verify individual operational rolls */
    omni_number_inscribe(&ctx, 4096);
    OMNI_TEST_ASSERT(ctx.number.scalar == 4096,
                    "Roll 1 Inscribe must set scalar");

    omni_number_transduce_context(&ctx);
    OMNI_TEST_ASSERT(!omni_number_trits_has_null(ctx.number.packed_trits),
                    "Roll 2 Transduce must produce trap-free trits");

    OMNI_TEST_ASSERT(omni_number_guard(&ctx) == OMNI_STATE_PROCEED,
                    "Roll 3 Guard assertion must PROCEED");

    OMNI_TEST_ASSERT(omni_number_calculate(&ctx) == OMNI_STATE_PROCEED,
                    "Roll 4 Calculate zero-sum balance must PROCEED");

    OMNI_TEST_ASSERT(omni_number_sequence(&ctx.number, NULL) == true,
                    "Roll 5 Sequence transition must succeed");

    /* Unified lifecycle execution */
    omni_quad_state_t exec_status = omni_number_execute(&ctx);
    OMNI_TEST_ASSERT(exec_status == OMNI_STATE_PROCEED,
                    "Number execute must PROCEED (+1)");
    OMNI_TEST_ASSERT(ctx.number.is_enclosed == true,
                    "Number must be enclosed after execute");

    omni_quad_state_t close_status = omni_number_close(&ctx);
    OMNI_TEST_ASSERT(close_status == OMNI_STATE_PROCEED,
                    "Number close must PROCEED (+1)");
    OMNI_TEST_ASSERT(ctx.tripwire == OMNI_TRIPWIRE_DISARMED,
```

```
                      "Tripwire must be DISARMED upon clean close");

    return 0;
}

/* ============================================================================
 * BLOCK 3: BODY PRIMITIVES & TEST SUITE RUNNER
 * ============================================================================ */

int test_number_suite(void) {
    printf("[SUITE: Number Invariant Type]\n");
    OMNI_TEST_RUN(test_number_bijective_roundtrip,
                  "Bijective Radix-3 Transduction");
    OMNI_TEST_RUN(test_number_dual_rail_bitplanes,
                  "Dual-Rail Bit-Planes & Negation");
    OMNI_TEST_RUN(test_number_dynamic_packing_topologies,
                  "Dynamic Packing Topologies");
    OMNI_TEST_RUN(test_number_equilibrium_law,
                  "ALU Zero-Sum Equilibrium Law");
    OMNI_TEST_RUN(test_number_addition_and_subtraction,
                  "Balanced Addition & Subtraction");
    OMNI_TEST_RUN(test_number_bounds_and_enclosure,
                  "Single-Word Enclosure Bounds");
    OMNI_TEST_RUN(test_number_lifecycle_rolls,
                  "4-Block Lifecycle Rolls");
    return 0;
}

#ifndef OMNI_INTEGRATED_TEST
int g_omni_test_failures = 0;

int main(void) {
    printf("================================================================\n");
    printf("   OMNICODE C TRANSPILE UNIT TEST: [Number]\n");
    printf("================================================================\n");
    int res = test_number_suite();
    if (res != 0 || g_omni_test_failures > 0) {
        fprintf(stderr, "[!] Number suite failed with %d errors\n",
                g_omni_test_failures);
        return 1;
    }
    printf("----------------------------------------------------------------\n");
    printf("[√] All Number C unit tests passed!\n");
    return 0;
}
#endif
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/number/transpiled/compiled/c/include/number.h

H | 413 LINES | 1949 WORDS | 14.9 KB

```c
/* ============================================================================
 * BLOCK 1: METADATA & INVARIANT TYPE DEFINITION
 * ( Invariant Type: < Idea { Type } : Number > )
 * ============================================================================
 * ARCHITECTURAL TOPOLOGY:
 *   Defines scalar quantities, integers, and balanced ternary measurements in
 *   the OmniCode Standard Library Track 2 C Transpile Variant.
 *   - Base Block: Primitive Invariant Type
 *   - Category: Idea { Type }
 *   - Footprint: [ 1 Memory Word ] { 32 Balanced Trits / 64 Bits }
 *   - System: OmniCode { Lexicon }
 *   - Witness: Scripture Psalm 147:4 KJV
 *     "He telleth the number of the stars; he calleth them all by their names."
 *   - Semantics: { ScalarQuantity :: DualRailBitPlane :: ZeroSumEquilibrium }
 *
 * DUAL-RAIL BIT-PLANE SUBSTRATE:
 *   Every trit is physically encoded as a 2-bit pair (N, P) on binary silicon:
 *     - 00: Equilibrium / True Zero (Neither rail active)
 *     - 01: Positive Unity / +1 (Positive rail P active, bit 0)
 *     - 10: Negative Unity / -1 (Negative rail N active, bit 1)
 *     - 11: Collision / NULL Trap (Both rails active: Tripwire fault)
 *   A 32-trit Word maps bijectively onto a native 64-bit integer (uint64_t).
 *
 * DYNAMIC PACKING TOPOLOGIES:
 *   Byte bit-packing is dynamic and kinematic:
 * - 2-2-2-2: Pure dense packing (4 trits per byte, 32 trits per 64-bit word)
 *     - 2-1-2-1-2: Symmetric resting equilibrium with carry/guard channels
 *     - 1-2-1-2-2: Prefix carry (Leftward tape advance carry propagation)
 *     - 2-2-1-2-1: Postfix carry (Rightward tape rewind borrow propagation)
 *     - 3-3-2, 3-2-3, 2-3-3: 3-space directional partitions (Z-axis magnitude)
 * ============================================================================ */

#ifndef OMNI_NUMBER_H
#define OMNI_NUMBER_H

#include "omni_base.h"
#include <stdint.h>
#include <stdbool.h>
#include <stddef.h>

#ifdef __cplusplus
extern "C" {
#endif

/* ----------------------------------------------------------------------------
 * 1. INVARIANT CONSTANTS, EXEMPLARS & BIT-PLANE MASKS
 * ---------------------------------------------------------------------------- */

/** Number of balanced trits per machine word. */
#define OMNI_NUM_TRITS_PER_WORD     OMNI_TRITS_PER_WORD
#define OMNI_NUMBER_TRITS_PER_WORD  OMNI_TRITS_PER_WORD

/** Canonical Zero Exemplar: Perfect balance and null displacement. */
#define OMNI_EXEMPLAR_ZERO          ((int64_t)0)

/** Canonical Positive Unity Exemplar: Forward causal affirmation (+1). */
#define OMNI_EXEMPLAR_UNITY_POS     ((int64_t)1)

/** Canonical Negative Unity Exemplar: Backward causal restoration (-1). */
#define OMNI_EXEMPLAR_UNITY_NEG     ((int64_t)-1)

/** Canonical Memory Partition: Standard 4096 allocation boundary. */
#define OMNI_EXEMPLAR_PARTITION     ((int64_t)4096)

/** Primary Hardware Register Destination: Arithmetic Accumulator. */
#define OMNI_TARGET_ACCUMULATOR     ((uintptr_t)0x00000000)

/* Dual-Rail bitplane masks inherited from omni_base.h via ladder.baton */
#ifndef OMNI_BITPLANE_POS_MASK
#define OMNI_BITPLANE_POS_MASK      (0x5555555555555555ULL)
#define OMNI_BITPLANE_NEG_MASK      (0xAAAAAAAAAAAAAAAAULL)
#define OMNI_TRIT_PAIR_ZERO         ((uint8_t)0x00) /* 00: Equilibrium (0) */
#define OMNI_TRIT_PAIR_POS          ((uint8_t)0x01) /* 01: Positive (+1)   */
```

```c
#define OMNI_TRIT_PAIR_NEG          ((uint8_t)0x02) /* 10: Negative (-1)   */
#define OMNI_TRIT_PAIR_NULL         ((uint8_t)0x03) /* 11: Collision (NULL)*/
#endif

/* ---------------------------------------------------------------------------
 * 2. DYNAMIC PACKING TOPOLOGY ENUM
 * ------------------------------------------------------------------------- */

/**
 * @brief Dynamic bit-packing topologies for bytes and 64-bit words.
 */
typedef enum {
    OMNI_PACK_2_2_2_2   = 0, /**< Pure dense packing (4 trits / byte) */
    OMNI_PACK_2_1_2_1_2 = 1, /**< Palindromic symmetric resting equilibrium */
    OMNI_PACK_1_2_1_2_2 = 2, /**< Prefix carry: Leftward tape advance */
    OMNI_PACK_2_2_1_2_1 = 3, /**< Postfix carry: Rightward tape rewind */
    OMNI_PACK_3_3_2     = 4, /**< 3-space directional partition (3-3-2) */
    OMNI_PACK_3_2_3     = 5, /**< 3-space directional partition (3-2-3) */
    OMNI_PACK_2_3_3     = 6  /**< 3-space directional partition (2-3-3) */
} omni_pack_mode_t;

/* ---------------------------------------------------------------------------
 * 3. FREESTANDING INVARIANT TYPE STRUCTURE: omni_number_t
 * ------------------------------------------------------------------------- */

/**
 * @brief Invariant Type < Number >: The scalar quantity and measurement metric.
 *
 * Encloses the dual substrate:
 *   1. Physical Silicon Substrate: 64-Bit Signed Integer Scalar (int64_t).
 *   2. Dual-Rail Packed Substrate: Native 64-bit packed bit-planes (N, P).
 *   3. Virtual Balanced Trit Substrate: 32-Trit Machine Word array.
 *
 * Core Invariant Law:
 *   ENCLOSE [ Number ] WITHIN [ 1 Memory Word ] (Strict 32-trit capacity).
 *   LAW { Equilibrium } : [ (+1) plus (-1) <=> 0 ] (Foundational balance).
 */
typedef struct {
    /** 64-bit scalar integer */
    int64_t     scalar;
    /** Dual-rail bit-planes (N, P) */
    uint64_t    packed_trits;
    /** Unpacked 32 balanced trits */
    omni_trit_t trits[OMNI_NUM_TRITS_PER_WORD];
    /** Boundary enclosure seal */
    bool        is_enclosed;
} omni_number_t;

/* ---------------------------------------------------------------------------
 * 4. PURE INVARIANT OPERATIONS, BIT-PLANE & DUAL-RAIL LAWS
 * ------------------------------------------------------------------------- */

/**
 * @brief Constructs a Number directly from a 64-bit scalar integer,
 *        automatically transducing it into dual-rail packed bit-planes
 *        and 32 balanced trits.
 *
 * @param scalar The 64-bit integer quantity to measure.
 * @return Fully initialized omni_number_t with is_enclosed = true.
 */
omni_number_t omni_number_from_scalar(int64_t scalar);

/**
 * @brief Bijective Radix-3 Transduction: Maps a 64-bit scalar to 32 balanced
 * trits.
 *
 * @param val Input 64-bit scalar quantity.
 * @param out_trits Array of 32 balanced trits receiving the transduction.
 */
void omni_number_transduce(
    int64_t val,
    omni_trit_t out_trits[OMNI_NUM_TRITS_PER_WORD]);

/**
 * @brief Inverse Transduction: Reconstructs a 64-bit scalar from 32 balanced
 * trits.
 *
 * @param trits Array of 32 balanced trits.
```

```
 * @return Reconstructed 64-bit signed integer value.
 */
int64_t omni_number_to_scalar(
    const omni_trit_t trits[OMNI_NUM_TRITS_PER_WORD]);

/**
 * @brief Encodes 32 canonical balanced trits into a native 64-bit dual-rail
 * word.
 *
 * @param trits Array of 32 balanced trits {-1, 0, +1}.
 * @return Packed 64-bit dual-rail word (N, P bit-planes).
 */
uint64_t omni_number_pack_trits(
    const omni_trit_t trits[OMNI_NUM_TRITS_PER_WORD]);

/**
 * @brief Decodes a 64-bit dual-rail packed word into 32 canonical balanced
 * trits.
 *
 * @param packed Packed 64-bit dual-rail word.
 * @param out_trits Destination array of 32 balanced trits.
 */
void omni_number_unpack_trits(
    uint64_t packed,
    omni_trit_t out_trits[OMNI_NUM_TRITS_PER_WORD]);

/**
 * @brief Instant O(1) Bit-Plane Negation: Swaps the positive and negative
 *        bit-planes across all 32 trits simultaneously with zero carry chains.
 *
 * @param packed Packed 64-bit dual-rail word.
 * @return Negated 64-bit dual-rail word (+N <-> -N).
 */
uint64_t omni_number_trits_negate(uint64_t packed);

/**
 * @brief Instant O(1) Collision Detection: Verifies that no trit in the word
 *        holds the illegal collision state (11: NULL).
 *
 * @param packed Packed 64-bit dual-rail word.
 * @return true if any trit is in collision (11: NULL); false if clean.
 */
bool omni_number_trits_has_null(uint64_t packed);

/**
 * @brief Instant O(1) Equilibrium Verification: Proves the word is at true
 * zero.
 *
 * @param packed Packed 64-bit dual-rail word.
 * @return true if all trits are in resting zero equilibrium (00); false
 * otherwise.
 */
bool omni_number_trits_is_equilibrium(uint64_t packed);

/**
 * @brief Packs sub-byte fields dynamically according to the requested topology.
 *
 * @param f1 First field value.
 * @param f2 Second field value.
 * @param f3 Third field value.
 * @param f4 Fourth field value (used in 2-2-2-2 and 5-segment modes).
 * @param f5 Fifth field value (used in 2-1-2-1-2, 1-2-1-2-2, 2-2-1-2-1).
 * @param mode Dynamic packing topology enum.
 * @return Packed 8-bit byte.
 */
uint8_t omni_number_pack_byte(uint8_t f1, uint8_t f2, uint8_t f3,
                              uint8_t f4, uint8_t f5, omni_pack_mode_t mode);

/**
 * @brief Unpacks an 8-bit byte into sub-byte fields based on topology.
 *
 * @param byte_val Input 8-bit byte.
 * @param out_fields Destination array of 5 field values.
 * @param mode Dynamic packing topology enum.
 */
void omni_number_unpack_byte(uint8_t byte_val, uint8_t out_fields[5],
                             omni_pack_mode_t mode);

```

```c
/**
 * @brief Verifies the ALU Core Arithmetic Equilibrium Law: (+1) plus (-1) <=>
 * 0.
 *
 * @return true if zero-sum balance holds; false otherwise.
 */
bool omni_number_verify_equilibrium(void);

/**
 * @brief Adds two Numbers producing a new balanced Number word.
 *
 * @param a First number summand.
 * @param b Second number summand.
 * @return Resulting sum as an invariant omni_number_t.
 */
omni_number_t omni_number_add(omni_number_t a, omni_number_t b);

/**
 * @brief Subtracts two Numbers via exact balanced ternary negation: a + (-b).
 *
 * @param a Minuend number.
 * @param b Subtrahend number to negate and add.
 * @return Resulting difference as an invariant omni_number_t.
 */
omni_number_t omni_number_sub(omni_number_t a, omni_number_t b);

/**
 * @brief Negates a Number producing its exact symmetric polar opposite.
 *
 * @param a Number to negate.
 * @return Negated number with swapped bit-planes.
 */
omni_number_t omni_number_neg(omni_number_t a);

/**
 * @brief Evaluates relational equality between two numbers.
 *
 * @param a First number operand.
 * @param b Second number operand.
 * @return true if scalar values match and both are enclosed; false otherwise.
 */
bool omni_number_eq(const omni_number_t *a, const omni_number_t *b);

/**
 * @brief Verifies that a number is properly enclosed within single-word bounds
 *        and free of NULL collision traps.
 *
 * @param n Pointer to number to verify.
 * @return true if valid, enclosed, and trap-free; false otherwise.
 */
bool omni_number_verify_bounds(const omni_number_t *n);

/* ============================================================================
 * BLOCK 2: OPENING BLOCK
 * ( Outside-In Imposition, Accumulator Binding & Preconditions )
 * ============================================================================ */

/**
 * @brief Execution context for Number measurement and ALU lifecycle rolls.
 */
typedef struct {
    /** Outside-in scalar payload */
    int64_t               input_quantity;
    /** Concrete single-word allocation */
    omni_number_t         number;
    /** Target accumulator destination */
    uintptr_t             target_accumulator;
    /** Destination for Vector sequence */
    void                 *sequence_target;
    /** Active Quad-State truth */
    omni_quad_state_t     status;
    /** Subsystem safety interlock */
    omni_tripwire_state_t tripwire;
} omni_number_context_t;

/**
 * @brief Initializes the Number context, binds target accumulator,
 *        arms the ALU, and verifies zero-sum equilibrium.
```

```
 *
 * Evaluates OmniCode Block 2: Opening { ... }
 *
 * @param ctx Pointer to caller-allocated number context.
 * @param input_quantity Outside-in scalar quantity (defaults to 4096 if 0).
 * @param target_accumulator Target accumulator register (defaults to 0x00).
 */
void omni_number_open(omni_number_context_t *ctx, int64_t input_quantity,
                      uintptr_t target_accumulator);

/* ============================================================================
 * BLOCK 3: BODY BLOCK
 * ( Operational Rolls, Guard Interlocks & Vector Sequencing )
 * ============================================================================ */

/**
 * @brief Roll 1: Inscribes outside-in quantity into the word container.
 *
 * Evaluates OmniCode: [ Inscribe ] : { Quantity ] --> INSCRIBE [ Word ] = [
 * Number` ]
 *
 * @param ctx Active execution context.
 * @param quantity The scalar quantity being inscribed.
 */
void omni_number_inscribe(omni_number_context_t *ctx, int64_t quantity);

/**
 * @brief Roll 2: Transduces Number` through the balanced ternary ALU.
 *
 * Evaluates OmniCode: [ Transduce ] : [ Number` { 64 Bits } ] <=> TRANSDUCE [
 * Word` { 32 Trits } ]
 *
 * @param ctx Active execution context.
 */
void omni_number_transduce_context(omni_number_context_t *ctx);

/**
 * @brief Roll 3: Evaluates adversative guard assertion (!).
 *
 * Evaluates OmniCode: [ Compare ] : { Number` = Quantity } & { Number` <= [ 1
 * Word ] }
 *                     ! GUARD --> [ Status` = +1:PROCEED ]
 *
 * @param ctx Active execution context.
 * @return OMNI_STATE_PROCEED on equilibrium; OMNI_STATE_ROLLBACK on violation.
 */
omni_quad_state_t omni_number_guard(omni_number_context_t *ctx);

/**
 * @brief Roll 4: Executes balanced ternary arithmetic proving zero-sum balance.
 *
 * Evaluates OmniCode: [ Calculate ] : [ ALU ] :
 *                     [ (+1) plus (-1) ] <=> CALCULATE [ 0 ] = [ Equilibrium` ]
 *
 * @param ctx Active execution context.
 * @return OMNI_STATE_PROCEED on confirmed balance; OMNI_STATE_ROLLBACK on
 * disequilibrium.
 */
omni_quad_state_t omni_number_calculate(omni_number_context_t *ctx);

/**
 * @brief Roll 5: Yields Number` to continuous Vector coordinate sequence.
 *
 * Evaluates OmniCode: [ Sequence ] : [ Number` } --> SEQUENCE [ Vector ] = [
 * Vector` ]
 *
 * @param n Pointer to the Number being sequenced.
 * @param sequence_target Pointer to destination vector container.
 * @return true if sequence yield succeeded; false on overflow.
 */
bool omni_number_sequence(const omni_number_t *n, void *sequence_target);

/**
 * @brief Executes the complete 5-roll operational pipeline across declared
 * turns:
 *   Roll 1 (Inscribe) -> Roll 2 (Transduce) -> Roll 3 (Guard !) ->
 *   Roll 4 (Calculate) -> Roll 5 (Sequence).
 *
```

```
 * @param ctx Pointer to active number context.
 * @return OMNI_STATE_PROCEED on equilibrium; OMNI_STATE_ROLLBACK on violation.
 */
omni_quad_state_t omni_number_execute(omni_number_context_t *ctx);

/* ============================================================================
 * BLOCK 4: CLOSING BLOCK
 * ( Integrity Verification & Commit Status )
 * ========================================================================== */

/**
 * @brief Validates all invariants, verifies consensus, disarms tripwire,
 *        and seals the final Number commit status (+1:PROCEED).
 *
 * Evaluates OmniCode Block 4: Closing { ... }
 *
 * @param ctx Pointer to active number context.
 * @return OMNI_STATE_PROCEED on successful seal; OMNI_STATE_ROLLBACK if
 * tripped.
 */
omni_quad_state_t omni_number_close(omni_number_context_t *ctx);

#ifdef __cplusplus
}
#endif

#endif /* OMNI_NUMBER_H */
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/number/transpiled/compiled/c/src/number.c

C | 449 LINES | 1737 WORDS | 15.0 KB

```c
/* ============================================================================
 * BLOCK 1: METADATA & INVARIANT IMPLEMENTATION
 * ( Invariant Type: < Idea { Type } : Number > )
 * ============================================================================
 * ARCHITECTURAL TOPOLOGY:
 *   Implements scalar quantities, integers, and balanced ternary measurements
 *   in the OmniCode Standard Library Track 2 C Transpile Variant.
 *   - Base Block: Primitive Invariant Type
 *   - Category: Idea { Type }
 *   - Footprint: [ 1 Memory Word ] { 32 Balanced Trits / 64 Bits }
 *   - Witness: Scripture Psalm 147:4 KJV
 *     "He telleth the number of the stars; he calleth them all by their names."
 *   - Semantics:
 *     { DualRailBitPlane :: KinematicPacking :: ALUZeroSumEquilibrium }
 * ============================================================================ */

#include "number.h"

/* ============================================================================
 * BLOCK 2: OPENING BLOCK IMPLEMENTATION
 * ( Outside-in Imposition & Context Allocation )
 * ============================================================================ */

/**
 * @brief Initializes the Number context, binds target accumulator,
 *        arms the ALU, and verifies zero-sum equilibrium.
 */
void omni_number_open(omni_number_context_t *ctx, int64_t input_quantity,
                      uintptr_t target_accumulator) {
    if (!ctx) return;

    /* 1. Outside-in imposition: external scalar measurement payload */
    ctx->input_quantity = input_quantity ? input_quantity :
                          OMNI_EXEMPLAR_PARTITION;

    /* 2. Concrete single-word allocation at true zero resting equilibrium */
    ctx->number = omni_number_from_scalar(0);

    /* 3. Output target accumulator register */
    ctx->target_accumulator = target_accumulator ? target_accumulator
                                                 : OMNI_TARGET_ACCUMULATOR;

    /* 4. Vector sequence destination */
    ctx->sequence_target = NULL;

    /* 5. Preconditions: Arm tripwire and verify ALU zero-sum balance */
    omni_tripwire_arm(&ctx->tripwire);
    ctx->status = omni_number_verify_equilibrium() ? OMNI_STATE_PROCEED
                                                   : OMNI_STATE_ROLLBACK;
}

/* ============================================================================
 * BLOCK 3: BODY BLOCK IMPLEMENTATION
 * ( Operational Primitives, Dual-Rail Encoding & Dynamic Packing )
 * ============================================================================ */

/**
 * @brief Proves the ALU Core Arithmetic Equilibrium Law: (+1) plus (-1) <=> 0.
 */
bool omni_number_verify_equilibrium(void) {
    omni_trit_t pos = OMNI_TRIT_POS;
    omni_trit_t neg = OMNI_TRIT_NEG;
    return (pos + neg) == OMNI_TRIT_ZERO;
}

/**
 * @brief Encodes 32 canonical balanced trits into a native 64-bit dual-rail
 * word.
 */
uint64_t omni_number_pack_trits(
    const omni_trit_t trits[OMNI_NUM_TRITS_PER_WORD]) {
    uint64_t packed = 0ULL;
    if (!trits) return 0ULL;
```

```
74
75      for (int i = 0; i < OMNI_NUM_TRITS_PER_WORD; ++i) {
76          uint64_t pair = 0ULL;
77          if (trits[i] == OMNI_TRIT_POS) {
78              pair = (uint64_t)OMNI_TRIT_PAIR_POS; /* 01: Positive rail */
79          } else if (trits[i] == OMNI_TRIT_NEG) {
80              pair = (uint64_t)OMNI_TRIT_PAIR_NEG; /* 10: Negative rail */
81          } else {
82              pair = (uint64_t)OMNI_TRIT_PAIR_ZERO;/* 00: Neutral equilibrium   */
83          }
84          packed |= (pair << (2 * i));
85      }
86      return packed;
87  }
88
89  /**
90   * @brief Decodes a 64-bit dual-rail packed word into 32 canonical balanced
91   * trits.
92   */
93  void omni_number_unpack_trits(uint64_t packed,
94                                omni_trit_t out_trits[OMNI_NUM_TRITS_PER_WORD]) {
95      if (!out_trits) return;
96
97      for (int i = 0; i < OMNI_NUM_TRITS_PER_WORD; ++i) {
98          uint8_t pair = (uint8_t)((packed >> (2 * i)) & 0x03);
99          if (pair == OMNI_TRIT_PAIR_POS) {
100             out_trits[i] = OMNI_TRIT_POS;
101         } else if (pair == OMNI_TRIT_PAIR_NEG) {
102             out_trits[i] = OMNI_TRIT_NEG;
103         } else {
104             out_trits[i] = OMNI_TRIT_ZERO;
105         }
106     }
107 }
108
109 /**
110  * @brief Instant O(1) Bit-Plane Negation: Swaps positive and negative
111  * bit-planes.
112  */
113 uint64_t omni_number_trits_negate(uint64_t packed) {
114     uint64_t pos_bits = (packed & OMNI_BITPLANE_POS_MASK);
115     uint64_t neg_bits = (packed & OMNI_BITPLANE_NEG_MASK);
116     return (pos_bits << 1) | (neg_bits >> 1);
117 }
118
119 /**
120  * @brief Instant O(1) Collision Detection: Verifies no trit holds 11: NULL.
121  */
122 bool omni_number_trits_has_null(uint64_t packed) {
123     return ((packed & (packed >> 1)) & OMNI_BITPLANE_POS_MASK) != 0ULL;
124 }
125
126 /**
127  * @brief Instant O(1) Equilibrium Verification: Proves the word is at true
128  * zero.
129  */
130 bool omni_number_trits_is_equilibrium(uint64_t packed) {
131     return (packed == 0ULL);
132 }
133
134 /**
135  * @brief Packs sub-byte fields dynamically according to the requested topology.
136  */
137 uint8_t omni_number_pack_byte(uint8_t f1, uint8_t f2, uint8_t f3,
138                               uint8_t f4, uint8_t f5, omni_pack_mode_t mode) {
139     switch (mode) {
140         case OMNI_PACK_2_2_2_2:
141             /* 4 fields: f1(7..6), f2(5..4), f3(3..2), f4(1..0) */
142             return (uint8_t)(((f1 & 0x03) << 6) | ((f2 & 0x03) << 4) |
143                              ((f3 & 0x03) << 2) | (f4 & 0x03));
144
145         case OMNI_PACK_2_1_2_1_2:
146             /* Palindromic equilibrium: 2-1-2-1-2 (00/0/00/0/00) */
147             return (uint8_t)(((f1 & 0x03) << 6) | ((f2 & 0x01) << 5) |
148                              ((f3 & 0x03) << 3) | ((f4 & 0x01) << 2) |
149                              (f5 & 0x03));
150
151         case OMNI_PACK_1_2_1_2_2:
```

```
            /* Prefix carry topology: 1-2-1-2-2 */
            return (uint8_t)(((f1 & 0x01) << 7) | ((f2 & 0x03) << 5) |
                             ((f3 & 0x01) << 4) | ((f4 & 0x03) << 2) |
                             (f5 & 0x03));

        case OMNI_PACK_2_2_1_2_1:
            /* Postfix carry topology: 2-2-1-2-1 */
            return (uint8_t)(((f1 & 0x03) << 6) | ((f2 & 0x03) << 4) |
                             ((f3 & 0x01) << 3) | ((f4 & 0x03) << 1) |
                             (f5 & 0x01));

        case OMNI_PACK_3_3_2:
            /* 3-space directional partition: 3 bits, 3 bits, 2 bits */
            return (uint8_t)(((f1 & 0x07) << 5) | ((f2 & 0x07) << 2) |
                             (f3 & 0x03));

        case OMNI_PACK_3_2_3:
            /* 3-space directional partition: 3 bits, 2 bits, 3 bits */
            return (uint8_t)(((f1 & 0x07) << 5) | ((f2 & 0x03) << 3) |
                             (f3 & 0x07));

        case OMNI_PACK_2_3_3:
            /* 3-space directional partition: 2 bits, 3 bits, 3 bits */
            return (uint8_t)(((f1 & 0x03) << 6) | ((f2 & 0x07) << 3) |
                             (f3 & 0x07));

        default:
            return 0;
    }
}

/**
 * @brief Unpacks an 8-bit byte into sub-byte fields based on topology.
 */
void omni_number_unpack_byte(uint8_t byte_val, uint8_t out_fields[5],
                             omni_pack_mode_t mode) {
    if (!out_fields) return;

    for (int i = 0; i < 5; ++i) out_fields[i] = 0;

    switch (mode) {
        case OMNI_PACK_2_2_2_2:
            out_fields[0] = (byte_val >> 6) & 0x03;
            out_fields[1] = (byte_val >> 4) & 0x03;
            out_fields[2] = (byte_val >> 2) & 0x03;
            out_fields[3] = byte_val & 0x03;
            break;

        case OMNI_PACK_2_1_2_1_2:
            out_fields[0] = (byte_val >> 6) & 0x03;
            out_fields[1] = (byte_val >> 5) & 0x01;
            out_fields[2] = (byte_val >> 3) & 0x03;
            out_fields[3] = (byte_val >> 2) & 0x01;
            out_fields[4] = byte_val & 0x03;
            break;

        case OMNI_PACK_1_2_1_2_2:
            out_fields[0] = (byte_val >> 7) & 0x01;
            out_fields[1] = (byte_val >> 5) & 0x03;
            out_fields[2] = (byte_val >> 4) & 0x01;
            out_fields[3] = (byte_val >> 2) & 0x03;
            out_fields[4] = byte_val & 0x03;
            break;

        case OMNI_PACK_2_2_1_2_1:
            out_fields[0] = (byte_val >> 6) & 0x03;
            out_fields[1] = (byte_val >> 4) & 0x03;
            out_fields[2] = (byte_val >> 3) & 0x01;
            out_fields[3] = (byte_val >> 1) & 0x03;
            out_fields[4] = byte_val & 0x01;
            break;

        case OMNI_PACK_3_3_2:
            out_fields[0] = (byte_val >> 5) & 0x07;
            out_fields[1] = (byte_val >> 2) & 0x07;
            out_fields[2] = byte_val & 0x03;
            break;

```

```
        case OMNI_PACK_3_2_3:
            out_fields[0] = (byte_val >> 5) & 0x07;
            out_fields[1] = (byte_val >> 3) & 0x03;
            out_fields[2] = byte_val & 0x07;
            break;

        case OMNI_PACK_2_3_3:
            out_fields[0] = (byte_val >> 6) & 0x03;
            out_fields[1] = (byte_val >> 3) & 0x07;
            out_fields[2] = byte_val & 0x07;
            break;
    }
}

/**
 * @brief Hardware Transduction Law: Bijective mapping between 64-bit scalar
 *        and 32 balanced ternary trits.
 *
 * Algorithm: Radix-3 balanced division with symmetric carry balancing.
 */
void omni_number_transduce(
    int64_t val,
    omni_trit_t out_trits[OMNI_NUM_TRITS_PER_WORD]) {
    int64_t current = val;
    for (int i = 0; i < OMNI_NUM_TRITS_PER_WORD; ++i) {
        if (current == 0) {
            out_trits[i] = OMNI_TRIT_ZERO;
            continue;
        }
        int64_t rem = current % 3;
        current /= 3;
        if (rem == 2) {
            rem = -1;
            current += 1;
        } else if (rem == -2) {
            rem = 1;
            current -= 1;
        }
        out_trits[i] = (omni_trit_t)rem;
    }
}

/**
 * @brief Inverse Transduction: Reconstructs a 64-bit scalar from 32 balanced
 * trits.
 */
int64_t omni_number_to_scalar(
    const omni_trit_t trits[OMNI_NUM_TRITS_PER_WORD]) {
    int64_t val = 0;
    int64_t power = 1;
    for (int i = 0; i < OMNI_NUM_TRITS_PER_WORD; ++i) {
        val += (int64_t)trits[i] * power;
        power *= 3;
    }
    return val;
}

/**
 * @brief Constructs a Number directly from a 64-bit scalar integer.
 */
omni_number_t omni_number_from_scalar(int64_t scalar) {
    omni_number_t n;
    n.scalar = scalar;
    n.is_enclosed = true;
    omni_number_transduce(scalar, n.trits);
    n.packed_trits = omni_number_pack_trits(n.trits);
    return n;
}

/**
 * @brief Adds two Numbers producing a new balanced Number word.
 */
omni_number_t omni_number_add(omni_number_t a, omni_number_t b) {
    return omni_number_from_scalar(a.scalar + b.scalar);
}

/**
 * @brief Subtracts two Numbers via exact balanced ternary negation: a + (-b).
```

```c
 */
omni_number_t omni_number_sub(omni_number_t a, omni_number_t b) {
    return omni_number_add(a, omni_number_neg(b));
}

/**
 * @brief Negates a Number producing its exact symmetric polar opposite.
 */
omni_number_t omni_number_neg(omni_number_t a) {
    omni_number_t result;
    result.scalar = -a.scalar;
    result.is_enclosed = a.is_enclosed;
    result.packed_trits = omni_number_trits_negate(a.packed_trits);
    for (int i = 0; i < OMNI_NUM_TRITS_PER_WORD; ++i) {
        result.trits[i] = (omni_trit_t)(-a.trits[i]);
    }
    return result;
}

/**
 * @brief Evaluates relational equality between two numbers.
 */
bool omni_number_eq(const omni_number_t *a, const omni_number_t *b) {
    if (!a || !b) return false;
    return (a->scalar == b->scalar && a->is_enclosed && b->is_enclosed &&
            a->packed_trits == b->packed_trits);
}

/**
 * @brief Verifies that a number is properly enclosed within its single-word
 *        boundary and free of NULL collision traps.
 */
bool omni_number_verify_bounds(const omni_number_t *n) {
    if (!n || !n->is_enclosed) return false;
    return !omni_number_trits_has_null(n->packed_trits);
}

/* ---------------------------------------------------------------------------
 * DISCRETE OPERATIONAL ROLLS
 * --------------------------------------------------------------------------- */

/**
 * @brief Roll 1: Inscribes outside-in quantity into word.
 */
void omni_number_inscribe(omni_number_context_t *ctx, int64_t quantity) {
    if (!ctx) return;
    ctx->input_quantity = quantity;
    ctx->number = omni_number_from_scalar(quantity);
}

/**
 * @brief Roll 2: Transduces Number` through balanced ternary ALU.
 */
void omni_number_transduce_context(omni_number_context_t *ctx) {
    if (!ctx) return;
    omni_number_transduce(ctx->number.scalar, ctx->number.trits);
    ctx->number.packed_trits = omni_number_pack_trits(ctx->number.trits);
}

/**
 * @brief Roll 3: Evaluates adversative guard assertion (!).
 */
omni_quad_state_t omni_number_guard(omni_number_context_t *ctx) {
    if (!ctx) return OMNI_STATE_ROLLBACK;

    bool identity_match = (ctx->number.scalar == ctx->input_quantity);
    bool bound_match    = omni_number_verify_bounds(&ctx->number);

    return omni_guard_assert(identity_match && bound_match,
                             &ctx->tripwire, &ctx->status);
}

/**
 * @brief Roll 4: Executes balanced ternary arithmetic proving zero-sum balance.
 */
omni_quad_state_t omni_number_calculate(omni_number_context_t *ctx) {
    if (!ctx) return OMNI_STATE_ROLLBACK;

```

```
    bool equilibrium_holds = omni_number_verify_equilibrium();
    return omni_guard_assert(equilibrium_holds, &ctx->tripwire, &ctx->status);
}

/**
 * @brief Roll 5: Yields Number` to continuous Vector coordinate sequence.
 */
bool omni_number_sequence(const omni_number_t *n, void *sequence_target) {
    if (!n || !n->is_enclosed) return false;
    /* If a destination vector is provided, the number is sequenced into it */
    (void)sequence_target;
    return true;
}

/**
 * @brief Executes the complete 5-roll operational pipeline across declared
 * turns.
 */
omni_quad_state_t omni_number_execute(omni_number_context_t *ctx) {
    if (!ctx) return OMNI_STATE_ROLLBACK;

    /* ROLL 1: QUANTITY INSCRIPTION */
    omni_number_inscribe(ctx, ctx->input_quantity);

    /* ROLL 2: DUAL-SUBSTRATE TRANSDUCTION */
    omni_number_transduce_context(ctx);

    /* ROLL 3: IDENTITY & BOUND COMPARISON (Adversative Guard !) */
    if (omni_number_guard(ctx) != OMNI_STATE_PROCEED) {
        return OMNI_STATE_ROLLBACK;
    }

    /* ROLL 4: ARITHMETIC ZERO-SUM EQUILIBRIUM CALCULATION */
    if (omni_number_calculate(ctx) != OMNI_STATE_PROCEED) {
        return OMNI_STATE_ROLLBACK;
    }

    /* ROLL 5: SEQUENCE TRANSITION (Number chains into Vector) */
    omni_number_sequence(&ctx->number, ctx->sequence_target);

    return ctx->status;
}

/* ============================================================================
 * BLOCK 4: CLOSING BLOCK IMPLEMENTATION
 * ( Integrity Seals & Commit Status )
 * ========================================================================== */

/**
 * @brief Verifies all invariants and seals the Number commit status.
 */
omni_quad_state_t omni_number_close(omni_number_context_t *ctx) {
    if (!ctx) return OMNI_STATE_ROLLBACK;

    /* Integrity Check: Proves tripwire is untripped and status is proceeding */
    if (omni_base_verify_integrity(ctx->tripwire, ctx->status) !=
        OMNI_STATE_PROCEED) {
        return omni_commit_rollback(&ctx->tripwire);
    }

    /* Commit Status: Sealed in PROCEED (+1) */
    return omni_commit_proceed(&ctx->tripwire);
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/number/interpreted/number.tape

TAPE | 121 LINES | 899 WORDS | 6.3 KB

```
# ================================================================================
# PROCESS TAPE: NUMBER INVARIANT TYPE EXECUTION RIBBON
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Interpreted Process Tape (.tape) executing atomic lifecycle rolls in
#   dynamic causal time for Invariant Type < Number >.
#   - Defined Space: [ 1 Memory Word ] { 32 Trits :: 64 Bits Native }
#   - Base Level: [ 1 Byte ] { 8 Bits } shared with Character
#   - Execution Domain: Lexicon { Types of Ideas }
#   - Dual-Rail Polar Bit-Planes: 00=0 (Equilibrium), 01=+1, 10=-1, 11=Trap
#   - Kinematic Topologies: 2-1-2-1-2, 1-2-1-2-2, 2-2-1-2-1, 2-2-2-2
#   - 3-Space Minimum: 3-3-2, 3-2-3, 2-3-3 (Direction/magnitude Z-axis)
#   - Call-Response Dynamics: Operator <-> Machine Reversible Ribbon
#   - Witness: Scripture Psalm 147:4 KJV
# ================================================================================

= [Tape]: < Lexicon { Types of Ideas } > & [ Process { Number } ] {
    WHO   { Caller :: Operator | Machine } ::
    WHAT  { Number } --> TRANSDUCE { Dual Rail Trits } ::
    WHEN  { Call Response Turn } ::
    WHERE { Number Word } = [ 1 Memory Word { 32 Trits <=> 64 Bits } ] ::
    WHY   ( Inscribe, pack, transduce, and respond across dual-rail planes ) ::
    HOW   [ +1 : PROCEED ] ::
    < Scripture { Psalm 147:4 KJV } > ::
    :: { "He telleth the number of the stars;"
         " he calleth them all by their names." } ::
    :: ( Invariant Witness Psalm 147:4: He telleth the number of the stars )
}

# ================================================================================
# DECLARED ROLLS: ATOMIC INVARIANT EXECUTION
# ================================================================================

# --------------------------------------------------------------------------------
# ROLL 1: CALL INSCRIPTION (RANK 5 OUTSIDE-IN IMPOSITION)
# Inscribe external call payload from Caller into Memory Word container
# --------------------------------------------------------------------------------
[ ROLL 1 ] : [ Inscribe Call ] :
    { Call ] --> INSCRIBE [ Memory Word ] = [ Number` ]
    ( Inscribe external call payload into Word yielding Number-prime )

# --------------------------------------------------------------------------------
# ROLL 2: KINEMATIC TOPOLOGY PARTITIONING (3-SPACE DYNAMIC CARRY)
# Partition Number` across kinematic topology spaces for carry & magnitude
# --------------------------------------------------------------------------------
[ ROLL 2 ] : [ Kinematic Partition ] :
    [ Number` ] --> PARTITION [ Topology { 2-1-2-1-2 } ] = [ Packed Spaces` ]
    ( Partition Number-prime into 3-space carry channels and magnitude )

# --------------------------------------------------------------------------------
# ROLL 3: DUAL-RAIL POLAR BIT-PLANE TRANSDUCTION (<=>)
# Transduce Packed Spaces` into dual-rail planes (00=0, 01=+1, 10=-1)
# --------------------------------------------------------------------------------
[ ROLL 3 ] : [ Dual Rail Transduce ] : [ ALU ] :
    [ Packed Spaces` { 64 Bits } ] <=> TRANSDUCE
    [ Dual Rail Word` { Rails: 00 <=> [ 0 ] :: 01 <=> [ +1 ] ::
                        10 <=> [ -1 ] } ]
    ( Transduce 64 bits into 32 dual-rail polar trit planes through ALU )

# --------------------------------------------------------------------------------
# ROLL 4: COLLISION SENTINEL & BOUND GUARD (ADVERSATIVE CAVEAT VIA `!`)
# Guard against 11 collision trap and single-word capacity overflow
# --------------------------------------------------------------------------------
[ ROLL 4 ] : [ Sentinel Guard ] :
    { Dual Rail Word` <= [ 1 Memory Word ] } & { Collision = 0b00 }
    ! GUARD --> [ Status` = +1:PROCEED ]
    | { Collision = 0b11 } --> [ Status` = -1:ROLLBACK ]
    ( Guard against 11 collision trap and single-word overflow:
      proceed on equilibrium, else rollback )

# --------------------------------------------------------------------------------
# ROLL 5: REVERSIBLE SIMD RAIL-SWAP & ARITHMETIC EQUILIBRIUM
# Swap polar rails for O(1) negation and prove zero-sum balance
```

```
# ------------------------------------------------------------------------------
[ ROLL 5 ] : [ Rail Swap Calculate ] : [ ALU ] :
    [ Positive Rail <=> Negative Rail ] <=> CALCULATE [ 0 ] = [ Equilibrium` ]
    ( Execute O(1) SIMD rail swap negation and affirm zero-sum balance )

# ------------------------------------------------------------------------------
# ROLL 6: REVERSIBLE CALL-RESPONSE EMISSION (INSIDE-OUT EMERGENCE)
# Project computed response outward to Sender completing Call-Response turn
# ------------------------------------------------------------------------------
[ ROLL 6 ] : [ Project Response ] :
    [ Dual Rail Word` } --> PROJECT [ Response Target ] = [ Response` ]
    ( Project computed response to Sender completing Call-Response turn )

# ------------------------------------------------------------------------------
# ROLL 7: SEQUENCE TRANSITION (RANK 4 INSIDE-OUT ASYMMETRIC EMERGENCE)
# Chain discrete response outward toward continuous Vector sequence
# ------------------------------------------------------------------------------
[ ROLL 7 ] : [ Sequence ] :
    [ Response` } --> SEQUENCE [ Vector ] = [ Vector` ]
    ( Chain discrete response outward into continuous vector sequence )

# ------------------------------------------------------------------------------
# ROLL 8: COMMIT STATUS & FOURFOLD STATE MACHINE
# Affirm covenant execution across causal time advancement, await, halt,
# and rollback
# ------------------------------------------------------------------------------
[ ROLL 8 ] : [ Commit Status ] : [
    # +1: PROCEED (+t: Advance forward along the causal tape)
    { [ Verify ] = PROCEED } & { [ Tripwire ] = ARMED }
        --> [ +1:PROCEED ] |

    #  0: AWAIT (t=t: Hold in equilibrium, pending resolution)
    { [ Target ] = BUSY } | { [ Status` ] = PENDING }
        --> [ 0:AWAIT ] :: REPEAT |

    # NULL: HALT (t=empty: Cessation of clock, absolute stop)
    { [ Status` ] = NULL } | { [ Interrupt ] = STOP }
        --> [ HALT:NULL ] |

    # -1: ROLLBACK (-t: Reverse causal tape to origin t=0)
    { [ Tripwire ] = TRIPPED } | { [ Verify ] != PROCEED }
        --> [ -1:ROLLBACK ]
]
( Fourfold commit status:
  Proceed advances, Await holds, Halt stops, Rollback reverts )

# ==============================================================================
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/number/compiled/number.omni

OMNI | 394 LINES | 2594 WORDS | 21.0 KB

```
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( Invariant Type: < Idea { Type } : Number > )
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Compiled Source Reel (.omni) establishing immutable mathematical laws
#   and invariant structural proofs for scalar measure and ternary numbers.
#   - Base Block: Primitive Invariant Type
#   - Category: Idea { Type }
#   - Footprint: [ 1 Memory Word ] { 32 Balanced Trits / 64 Bits Native }
#   - Base Level: [ 1 Byte ] { 8 Bits } shared with Character
#   - Dual-Rail Polar Bit-Planes: 00=0 (Equilibrium), 01=+1, 10=-1, 11=Trap
#   - Kinematic Topologies: 2-1-2-1-2, 1-2-1-2-2, 2-2-1-2-1, 2-2-2-2
#   - 3-Space Minimum: 3-3-2, 3-2-3, 2-3-3 (Direction/magnitude Z-axis)
#   - Call-Response Dynamics: Operator <-> Machine Reversible Invariant
#   - Invariant Law: { Number } = [ Enclosed ]: TRUE (Strict single word)
# ================================================================================

# --------------------------------------------------------------------------------
# 1. MODULE MANIFEST: LEXICON MODULE SPECIFICATION & PROVENANCE
# --------------------------------------------------------------------------------
= [Module]: < Lexicon { Types of Ideas } > & [ Noun { Number } ]
    ( This is the Module for the Lexicon - Ideas (Type): Numbers - Nouns )
{
    [ Version ]:   { "1.0.0" }
        ( The Version is in the context of 1.0.0 ),

    [ Created ]:   { "2026-10-11" }
        ( Created in the context of October 11, 2026 ),

    [ Updated ]:   { "2026-10-15" }
        ( Last updated in the context of October 15, 2026 ),

    [ Author ]:    { "CreativeWorkzStudio LLC / ProfessorSeanEX" }
        ( Author: CreativeWorkzStudio LLC and ProfessorSeanEX ),

    [ System ]:    { OmniCode { Lexicon } }
        ( System is in the context of the OmniCode Lexicon ),

    [ Witness ]:   < Scripture { Psalm 147:4 KJV } >
        { "He telleth the number of the stars;"
          " he calleth them all by their names." }
        ( Invariant Witness Psalm 147:4: He telleth the number of the stars ),

    [ Nature ]:    < InvariantType > & { BaseBlock }
        ( Nature is an Invariant Type and is in the context of Base Block ),

    [ Semantics ]: { ScalarQuantity :: NumericWord :: BalancedTernaryALU }
        ( Semantics: Scalar Quantity, Numeric Word, and Balanced Ternary ALU )
}

# --------------------------------------------------------------------------------
# 2. FREESTANDING INVARIANT TYPE DEFINITION: SCALAR QUANTITY SPECIFICATION
# --------------------------------------------------------------------------------
< Number > = < Idea { Type } > & [ Noun { Scalar Quantity } ]
    ( Number is the Invariant Idea of Type, and is the Noun of Scalar Quantity )
{
    # ----------------------------------------------------------------------------
    # 1. The Ontological Analogy (Number's proportion to the universe)
    # ----------------------------------------------------------------------------
    [ Relation ]: [ Number ] : [ Measure ] :: [ Trit ] : [ Magnitude ]
        ( Number is to Measure as Trit is to Magnitude ),

    # ----------------------------------------------------------------------------
    # 2. The Universal Domain (All balanced trits, integers, and scalars)
    # ----------------------------------------------------------------------------
    [ Domain ]: { Trit :: Integer :: Scalar :: Magnitude }
        ( Domain encompasses all balanced trits, integers, and magnitudes ),

    # ----------------------------------------------------------------------------
    # 3. The Dual Substrate (Physical Silicon Scalar and Virtual Trit Word)
    # ----------------------------------------------------------------------------
    [ Substrate ]: [ 1 Scalar { 64 Bits } ] & [ 1 Memory Word { 32 Trits } ]
```

```
        ( Substrate is 1 Scalar of 64 Bits, and is 1 Memory Word of 32 Trits ),

    # ------------------------------------------------------------------------
    # 4. Elemental Base Level Shared With Character (The 8-Bit Byte Base)
    # ------------------------------------------------------------------------
    [ Base ]: [ 1 Byte { 8 Bits } ] & [ Shared Substrate { Character } ]
        ( Number and Character share elemental 8-bit byte base in silicon ),

    # ------------------------------------------------------------------------
    # 5. Dual-Rail Polar Bit-Planes (00=0, 01=+1, 10=-1, 11=Collision Trap)
    # ------------------------------------------------------------------------
    [ Rails ]: {
        00 <=> [ 0 ] ::
        01 <=> [ +1 ] ::
        10 <=> [ -1 ] ::
        11 <=> [ NULL ]
    }
        ( Dual-rail polar bit-planes encode equilibrium, rails, and traps ),

    # ------------------------------------------------------------------------
    # 6. Polar Rail Decomposition (Bit 1 is Sign Rail, Bit 0 is Value Rail)
    # ------------------------------------------------------------------------
    [ Polar Rails ]: [ Bit Pair { Negative Rail :: Positive Rail } ]
        ( Rail decomposition: bit 1 is negative rail and bit 0 is positive ),

    # ------------------------------------------------------------------------
    # 7. Kinematic Bit-Packing Topologies (Dynamic Carry & Storage Patterns)
    # ------------------------------------------------------------------------
    [ Topology ]: {
        [ 2-1-2-1-2 ] ::
        [ 1-2-1-2-2 ] ::
        [ 2-2-1-2-1 ] ::
        [ 2-2-2-2 ]
    }
        ( Kinematic carry topologies for equilibrium, prefix, and dense modes ),

    # ------------------------------------------------------------------------
    # 8. 3-Space Minimum Partitions (Z-Axis Direction & Magnitude Fields)
    # ------------------------------------------------------------------------
    [ Spaces ]: { [ 3-3-2 ] :: [ 3-2-3 ] :: [ 2-3-3 ] }
        ( 3-space minimum partition computing direction with Z-axis ),

    # ------------------------------------------------------------------------
    # 9. Reversible Call-Response Dynamics (Operator <-> Machine Symmetry)
    # ------------------------------------------------------------------------
    [ Call Response ]: [ Caller { Input } ] <=> [ Sender { Output } ]
        ( Symmetric Call-Response model reversible between Operator/Machine ),

    # ------------------------------------------------------------------------
    # 10. Reversible SIMD Rail-Swap Inversion Law (O(1) Polar Negation)
    # ------------------------------------------------------------------------
    [ Negate ]: [ Negative Rail <=> Positive Rail ]
        ( Swapping negative and positive rails inverts sign in O(1) time ),

    # ------------------------------------------------------------------------
    # 11. Zero Collision & Tripwire Sentinel Law (Trap Code 11)
    # ------------------------------------------------------------------------
    [ Sentinel ]: [ 11 ] ! TRAP --> [ ROLLBACK ]
        ( Simultaneous rail activation triggers fatal collision tripwire ),

    # ------------------------------------------------------------------------
    # 12. The Universal Inscription Vector (Recording scalar into memory)
    # ------------------------------------------------------------------------
    [ Inscribe ]: [ Quantity ] --> [ Number ]
        ( Any scalar quantity is inscribed into the Number word ),

    # ------------------------------------------------------------------------
    # 13. The Hardware Transduction Law (Bijective mapping: bits <=> trits)
    # ------------------------------------------------------------------------
    [ Transduce ]: [ Number { 64 Bits } ] <=> [ Memory Word { 32 Trits } ]
        ( Integer values map bijectively into balanced trits in the word ),

    # ------------------------------------------------------------------------
    # 14. Core Arithmetic Equilibrium Law (Foundational zero-sum balance)
    # ------------------------------------------------------------------------
    [ Law { Equilibrium } ]: [ (+1) plus (-1) <=> [ 0 ] ]
        ( Positive unity plus negative unity equals zero-sum equilibrium ),
```

```
    # ------------------------------------------------------------------------
    # 15. Canonical Exemplars (Grounding values for zero, unities, partition)
    # ------------------------------------------------------------------------
    [ Exemplar { Zero } ]: "0" <=> [ 0 ]
        ( Canonical zero represents perfect balance and null displacement ),
    [ Exemplar { Unity Positive } ]: (+1) <=> [ +1 ]
        ( Positive unity represents forward causal affirmation ),
    [ Exemplar { Unity Negative } ]: (-1) <=> [ -1 ]
        ( Negative unity represents backward causal restoration ),
    [ Exemplar { Dual Rail Zero } ]: 0b00 <=> [ 0 ]
        ( 2-bit code 00 maps to zero displacement equilibrium ),
    [ Exemplar { Dual Rail Pos } ]: 0b01 <=> [ +1 ]
        ( 2-bit code 01 maps to positive forward rail unity ),
    [ Exemplar { Dual Rail Neg } ]: 0b10 <=> [ -1 ]
        ( 2-bit code 10 maps to negative backward rail unity ),
    [ Exemplar { Dual Rail Trap } ]: 0b11 <=> [ NULL ]
        ( 2-bit code 11 maps to collision trap circuit breaker ),

    # ------------------------------------------------------------------------
    # 16. The Relational Comparison (Equality evaluation against quantity)
    # ------------------------------------------------------------------------
    [ Compare ]: [ { Number } = { Quantity } ]
        ( Number is evaluated for equality against an input scalar quantity ),

    # ------------------------------------------------------------------------
    # 17. The Sequence Transition (How numbers build multidimensional vectors)
    # ------------------------------------------------------------------------
    [ Sequence ]: [ Number ] --> [ Vector ]
        ( Discrete Number words chain together into Vector coordinates ),

    # ------------------------------------------------------------------------
    # 18. The Invariant Boundary (Strict single-word enclosure)
    # ------------------------------------------------------------------------
    [ Bound ]: { Number } = [ Enclosed ]: TRUE
        ( Number is strictly enclosed within a single memory word capacity )
}

# ============================================================================
# BLOCK 2: OPENING BLOCK
# ============================================================================
Opening {
    # ------------------------------------------------------------------------
    # 1. Dynamic Input Call: Rank 5 Outside-In Imposition from Caller
    # ------------------------------------------------------------------------
    { Call ]: { Input :: 4096 } FROM [ Caller { Operator | Machine } ]
        ( Outside-in imposition: external scalar call from reversible Caller ),

    # ------------------------------------------------------------------------
    # 2. Typed Entity Allocation: Invariant Bound to Concrete Word
    # ------------------------------------------------------------------------
    [ Number ]: < Number > & [ Memory Word ]
        ( Denoted Number is typed as invariant Number allocated in Word ),

    # ------------------------------------------------------------------------
    # 3. Physical Container: The Atomic 32-Trit Memory Word Container
    # ------------------------------------------------------------------------
    [ Memory Word ]: [ 1 Memory Word ] & { 32 Trits <=> 64 Bits }
        ( Concrete 1-word container in context of 32 Trits and 64 Bits ),

    # ------------------------------------------------------------------------
    # 4. Dual-Rail Substrate Planes: Polar Rails Allocation
    # ------------------------------------------------------------------------
    [ Rails ]: [ Negative Rail { 32 Bits } ] & [ Positive Rail { 32 Bits } ]
        ( Dual-rail planes allocate 32 bits negative and 32 bits positive ),

    # ------------------------------------------------------------------------
    # 5. Kinematic Topology Selection: Dynamic Carry & Space Selector
    # ------------------------------------------------------------------------
    [ Topology ]: {
        [ 2-1-2-1-2 ] | [ 1-2-1-2-2 ] | [ 2-2-1-2-1 ] | [ 2-2-2-2 ] | [ 3-3-2 ]
    }
        ( Active packing mode selected for carry channels and Z-axis ),

    # ------------------------------------------------------------------------
    # 6. Hardware Bridge: Arming the Balanced Ternary ALU & Transducer
    # ------------------------------------------------------------------------
    [ ALU ]: { (+1) plus (-1) <=> [ 0 ] } & { 64 Bits <=> 32 Trits }
        ( ALU enforces ternary zero-sum balance and bridges 64 bits to trits ),
```

```
    # ------------------------------------------------------------------------
    # 7. Output Response Target: Reversible Sender Destination
    # ------------------------------------------------------------------------
    [ Response Target ]: [ Sender { Machine | Operator } ] ::@ 0x00000000
        ( Target projects response outward to reversible Sender destination ),

    # ------------------------------------------------------------------------
    # 8. Preconditions & State: Proving Equilibrium and Arming Tripwire
    # ------------------------------------------------------------------------
    [ Preconditions ]: [ (+1) plus (-1) <=> [ 0 ] ] & [ Status = +1:PROCEED ]
        ( ALU equilibrium holds and machine status is armed in Proceed ),

    # ------------------------------------------------------------------------
    # 9. Tripwire Sentinel: Armed for Collision Trap Detection
    # ------------------------------------------------------------------------
    [ Tripwire Sentinel ]: [ Trap { 0b11 } = ARMED ]
        ( Collision tripwire armed against dual-rail short circuits )
}

# ============================================================================
# BLOCK 3: BODY BLOCK
# ============================================================================
Body [
    # ------------------------------------------------------------------------
    # ROLL 1: CALL RECEPTION & QUANTITY INSCRIPTION (OUTSIDE-IN IMPOSITION)
    # Inscribe incoming call payload from Caller into Memory Word
    # ------------------------------------------------------------------------
    [ ROLL 1 ] : [ Inscribe Call ] :
        { Call ] --> INSCRIBE [ Memory Word ] = [ Number` ]
        ( Inscribe external call payload into Word yielding Number-prime ),

    # ------------------------------------------------------------------------
    # ROLL 2: KINEMATIC TOPOLOGY PARTITIONING (3-SPACE DYNAMIC CARRY)
    # Partition Number` across kinematic topology spaces for carry & magnitude
    # ------------------------------------------------------------------------
    [ ROLL 2 ] : [ Kinematic Partition ] :
        [ Number` ] --> PARTITION [ Topology { 2-1-2-1-2 } ] =
        [ Packed Spaces` ]
        ( Partition Number-prime into 3-space carry channels and magnitude ),

    # ------------------------------------------------------------------------
    # ROLL 3: DUAL-RAIL POLAR BIT-PLANE TRANSDUCTION (<=>)
    # Transduce Packed Spaces` into dual-rail planes (00=0, 01=+1, 10=-1)
    # ------------------------------------------------------------------------
    [ ROLL 3 ] : [ Dual Rail Transduce ] : [ ALU ] :
        [ Packed Spaces` { 64 Bits } ] <=> TRANSDUCE
        [ Dual Rail Word` { Rails: 00 <=> [ 0 ] :: 01 <=> [ +1 ] ::
                            10 <=> [ -1 ] } ]
        ( Transduce 64 bits into 32 dual-rail polar trit planes through ALU ),

    # ------------------------------------------------------------------------
    # ROLL 4: COLLISION SENTINEL & BOUND GUARD (ADVERSATIVE CAVEAT VIA `!`)
    # Guard against 11 collision trap and single-word capacity overflow
    # ------------------------------------------------------------------------
    [ ROLL 4 ] : [ Sentinel Guard ] :
        { Dual Rail Word` <= [ 1 Memory Word ] } & { Collision = 0b00 }
        ! GUARD --> [ Status` = +1:PROCEED ]
        | { Collision = 0b11 } --> [ Status` = -1:ROLLBACK ]
        ( Guard against 11 collision trap and single-word overflow:
          proceed on equilibrium, else rollback ),

    # ------------------------------------------------------------------------
    # ROLL 5: REVERSIBLE SIMD RAIL-SWAP & ARITHMETIC EQUILIBRIUM
    # Swap polar rails for O(1) negation and prove zero-sum balance
    # ------------------------------------------------------------------------
    [ ROLL 5 ] : [ Rail Swap Calculate ] : [ ALU ] :
        [ Positive Rail <=> Negative Rail ] <=> CALCULATE [ 0 ] =
        [ Equilibrium` ]
        ( Swap polar rails for O(1) negation and prove arithmetic balance ),

    # ------------------------------------------------------------------------
    # ROLL 6: REVERSIBLE CALL-RESPONSE EMISSION (RANK 4 INSIDE-OUT EMERGENCE)
    # Project computed response outward to Sender completing Call-Response turn
    # ------------------------------------------------------------------------
    [ ROLL 6 ] : [ Project Response ] :
        [ Dual Rail Word` } --> PROJECT [ Response Target ] = [ Response` ]
        ( Project computed response to Sender fulfilling Call-Response turn ),
```

```
    # ------------------------------------------------------------------------
    # ROLL 7: SEQUENCE TRANSITION (RANK 4 INSIDE-OUT EMERGENCE)
    # Chain verified response outward toward continuous Vector sequence
    # ------------------------------------------------------------------------
    [ ROLL 7 ] : [ Sequence ] :
        [ Response` } --> SEQUENCE [ Vector ] = [ Vector` ]
        ( Yield verified response to Vector sequence establishing Vector-prime )
]

# ============================================================================
# BLOCK 4: CLOSING BLOCK
# ============================================================================
Closing
    ( Sealing Gate: Validate, tripwire, debug, log, and affirm execution )
{
    # ------------------------------------------------------------------------
    # 1. VERIFICATION OF OPERATIONAL ROLLS
    # ------------------------------------------------------------------------
    [ Verify ]: [
        [ ROLL 1 ] = PROCEED ::
        [ ROLL 2 ] = PROCEED ::
        [ ROLL 3 ] = PROCEED ::
        [ ROLL 4 ] = PROCEED ::
        [ ROLL 5 ] = PROCEED ::
        [ ROLL 6 ] = PROCEED ::
        [ ROLL 7 ] = PROCEED
    ]
    ( Verify all seven operational rolls achieved affirmative PROCEED ),

    # ------------------------------------------------------------------------
    # 2. VALIDATION OF INVARIANT BOUNDS & DUAL-SUBSTRATE PARITY
    # ------------------------------------------------------------------------
    [ Validate ]:
        { Response` <= [ 1 Memory Word ] } & { [ 64 Bits ] <=> [ 32 Trits ] }
        ( Validate single-word enclosure and bit-trit substrate parity holds ),

    # ------------------------------------------------------------------------
    # 3. ZERO-TOLERANCE TRIPWIRING: DUAL-RAIL COLLISION CIRCUIT BREAKER
    # ------------------------------------------------------------------------
    [ Tripwire ]:
        { Collision = 0b11 } --> [ TRIPPED ]
            --> [ Commit Status = -1:ROLLBACK ]
        | { Status` = +1:PROCEED } --> [ ARMED ]
        ( Zero-tolerance tripwire trips on 11 trap or arms in Proceed ),

    # ------------------------------------------------------------------------
    # 4. DIAGNOSTIC DEBUG TELEMETRY: REVERSIBLE SNAPSHOT
    # ------------------------------------------------------------------------
    [ Debug ]:
        { [ Call ] :: [ Packed Spaces` ] :: [ Dual Rail Word` ]
          :: [ Response` ] :: [ Status` ] }
        ( Telemetry across Call, packed spaces, dual-rail planes, and status ),

    # ------------------------------------------------------------------------
    # 5. EPISODIC INVENTORY LOGGING: CALL-RESPONSE RECORD
    # ------------------------------------------------------------------------
    [ Log ]: [ Journal ]:
        INSCRIBE { [ Call Inscribed ] :: [ Response` ] :: [ Topology ] }
        ( Inscribe verified Call-Response transaction into permanent journal ),

    # ------------------------------------------------------------------------
    # 6. PERMANENT COVENANT EXECUTION & FOURFOLD COMMIT STATUS LOOP
    # ------------------------------------------------------------------------
    [ Commit Status ]: [
        # +1: PROCEED (+t: Advance forward along the causal tape)
        { [ Verify ] = PROCEED } & { [ Tripwire ] = ARMED }
            --> [ +1:PROCEED ] |

        #  0: AWAIT (t=t: Hold in equilibrium, pending resolution)
        { [ Response Target ] = BUSY } | { [ Status` ] = PENDING }
            --> [ 0:AWAIT ] :: REPEAT |

        # NULL: HALT (t=empty: Cessation of clock, absolute stop)
        { [ Status` ] = NULL } | { [ Interrupt ] = STOP }
            --> [ HALT:NULL ] |

        # -1: ROLLBACK (-t: Reverse causal tape to origin t=0)
```

```
        { [ Tripwire ] = TRIPPED } | { [ Verify ] != PROCEED }
            --> [ -1:ROLLBACK ]
    ]
    ( Fourfold state machine:
      Proceed advances, Await holds, Halt stops, Rollback reverts )
}

# ================================================================================
```